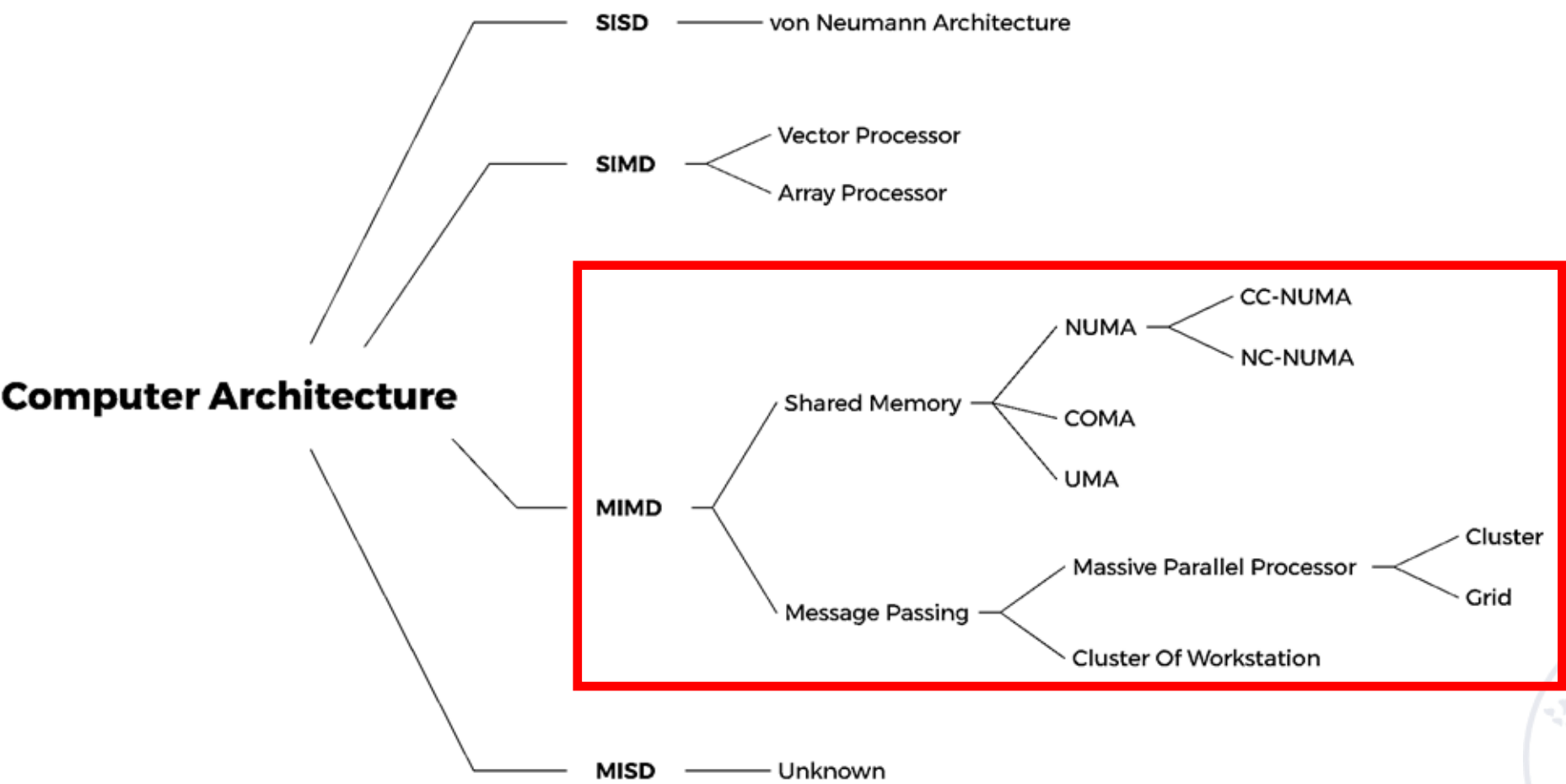


DLP and TLP



Flynn



The turning away from the conventional organization came in the middle 1960s, when the law of diminishing returns began to take effect in the effort to increase the operational speed of a computer. . . . Electronic circuits are ultimately limited in their speed of operation by the speed of light . . . and many of the circuits were already operating in the nanosecond range.

W. Jack Bouknight et al.,
The Illiac IV System (1972)



We are dedicating all of our future product development to multicore designs. We believe this is a key inflection point for the industry.

Intel President Paul Otellini,
*describing Intel's future direction at the
Intel Developer Forum in 2005*



Since 2004 processor designers have increased core counts to exploit Moore's Law scaling, rather than focusing on single-core performance. The failure of Dennard scaling, to which the shift to multicore parts is partially a response, may soon limit multicore scaling just as single-core scaling has been curtailed.

Hadi Esmaeilzadeh, et al.,
*Power Limitations and Dark Silicon
Challenge the Future of Multicore* (2012)



From ILP to TLP

- Thread-level parallelism is identified at a high level by software system or programmer;
- The threads consist of hundreds to millions of instructions that may be executed in parallel.



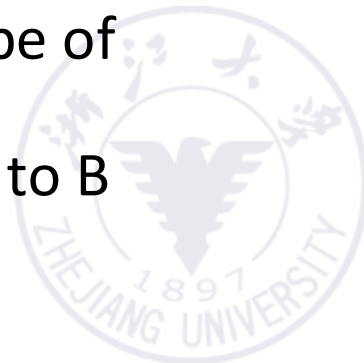
From TLP to MIMD Architecture

- TLP implies the existence of multiple program counters.
- Thus TLP is exploited primarily through MIMDs



MIMD Architecture

- Multi-processor system——based on **shared memory**
 - There is only a unique address space in the system, and all processors share this address space.
 - A unique address space does not mean that there is only one memory physically. The shared address space can be realized by a physically shared memory, or can be realized by a distributed memory with the support of hardware and software.
- Multi-computer system——based on **message passing**
 - Each processor has its own memory, which can only be accessed by the processor and cannot be directly accessed by other processors. This type of memory is called local memory or private memory.
 - When processor A needs to send data to processor B, A sends the data to B in the form of a message.

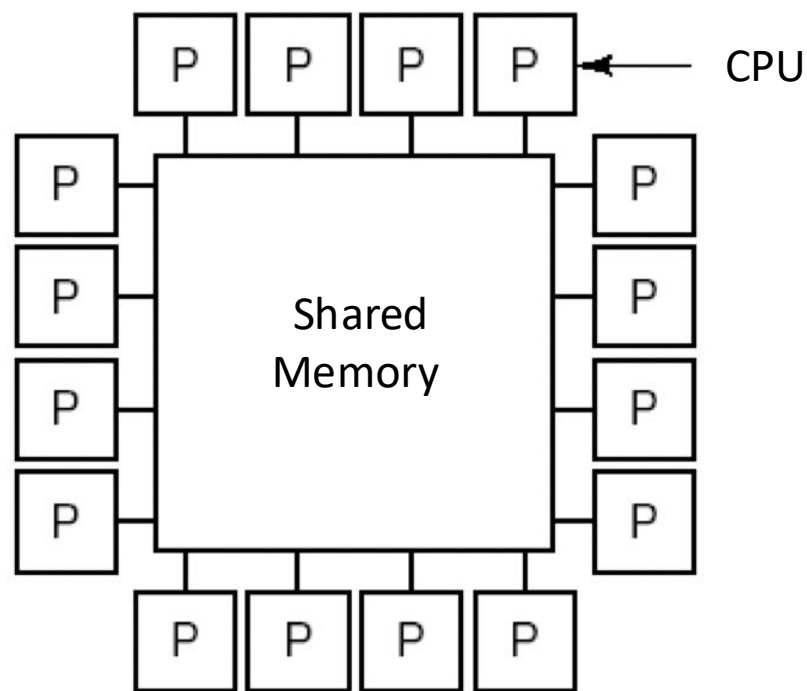


MIMD Architecture

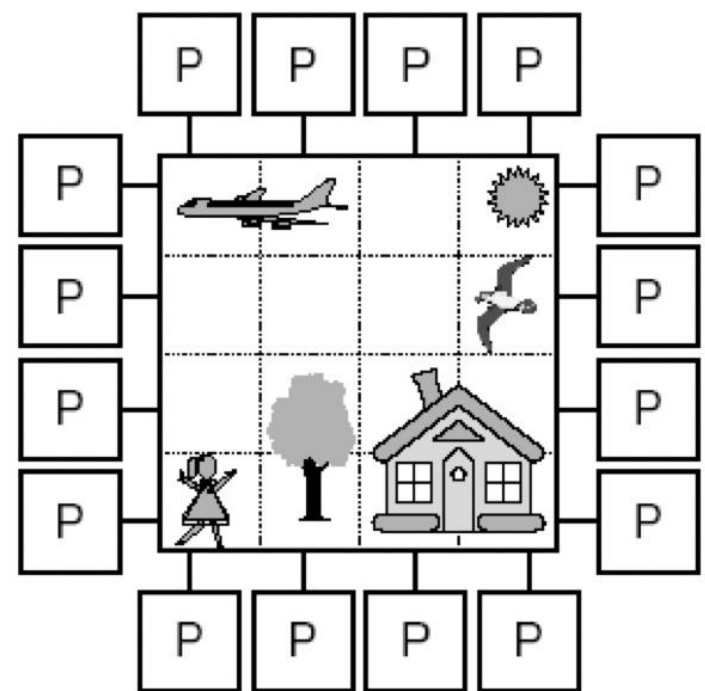
- Multi-processor system——based on **shared memory**
 - There is only a unique address space in the system, and all processors share this address space.
 - A unique address space does not mean that there is only one memory physically. The shared address space can be realized by a physically shared memory, or can be realized by a distributed memory with the support of hardware and software.
- Multi-computer system——based on message passing
 - Each processor has its own memory, which can only be accessed by the processor and cannot be directly accessed by other processors. This type of memory is called local memory or private memory.
 - When processor A needs to send data to processor B, A sends the data to B in the form of a message.



Shared Memory System



(a) A shared memory system with 16 CPUs

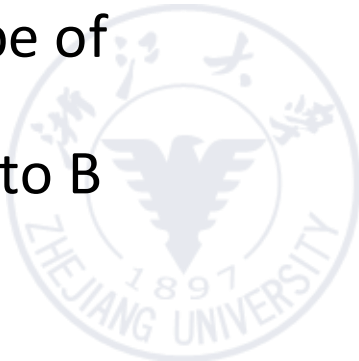


(b) A pictures is divided into 16 parts which are processed by different CPUs

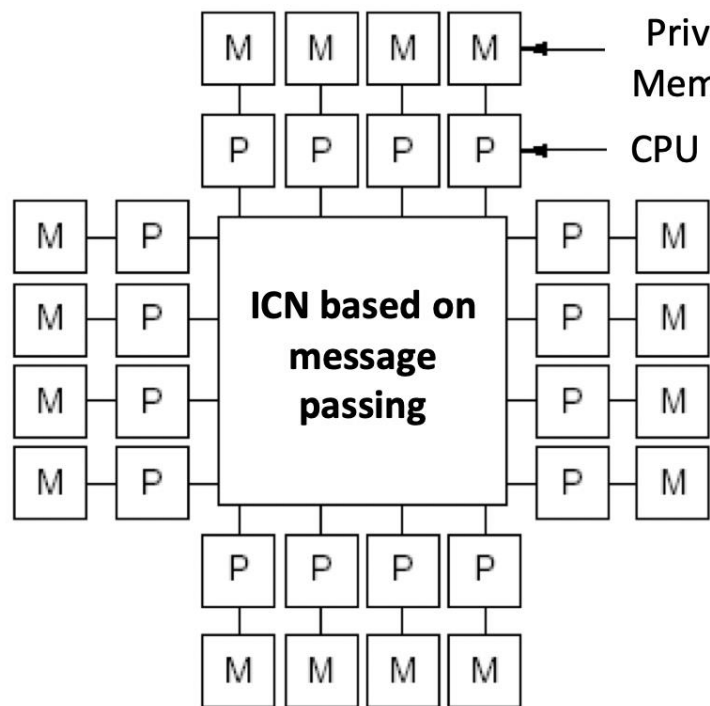


MIMD Architecture

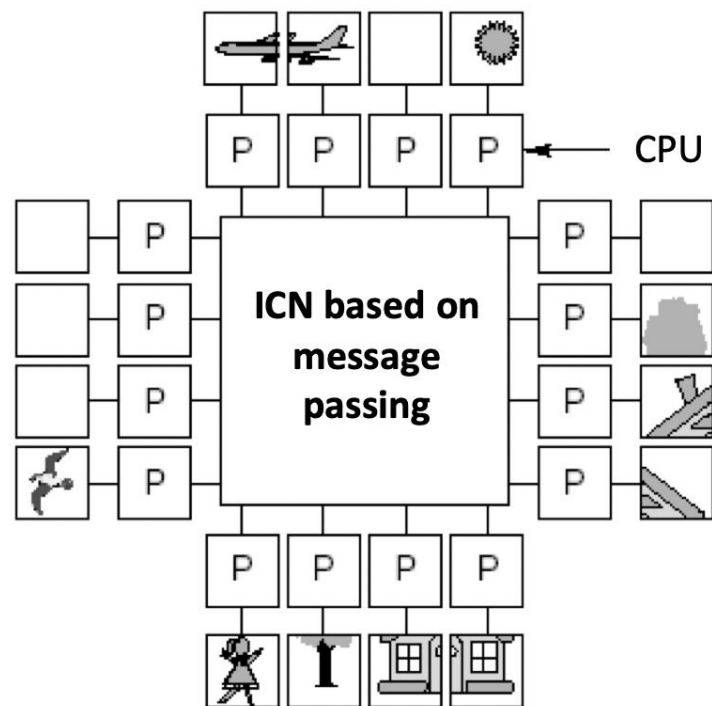
- Multi-processor system——based on shared memory
 - There is only a unique address space in the system, and all processors share this address space.
 - A unique address space does not mean that there is only one memory physically. The shared address space can be realized by a physically shared memory, or can be realized by a distributed memory with the support of hardware and software.
- Multi-computer system——based on **message passing**
 - Each processor has its own memory, which can only be accessed by the processor and cannot be directly accessed by other processors. This type of memory is called local memory or private memory.
 - When processor A needs to send data to processor B, A sends the data to B in the form of a message.



Multi-computer system based on message passing



(a) Every CPU owns its private memory

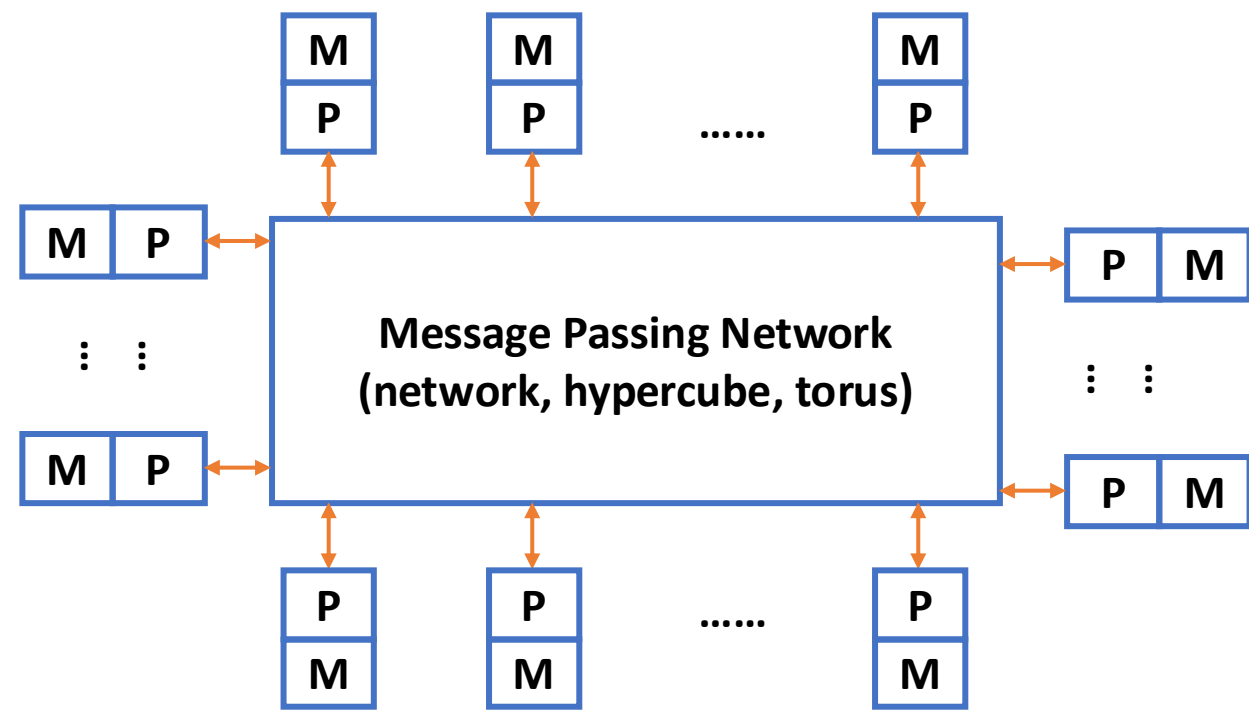


(b) A pictures is divided into 16 parts which are processed and stored separately



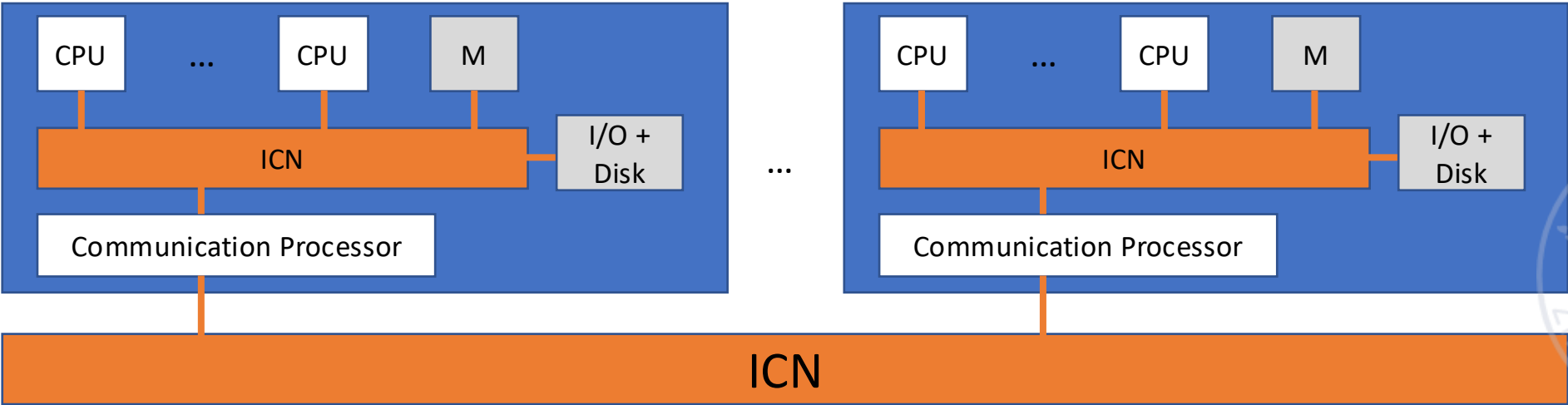
Multi-computer memory access model

- NORMA (No-Remote Memory Access)

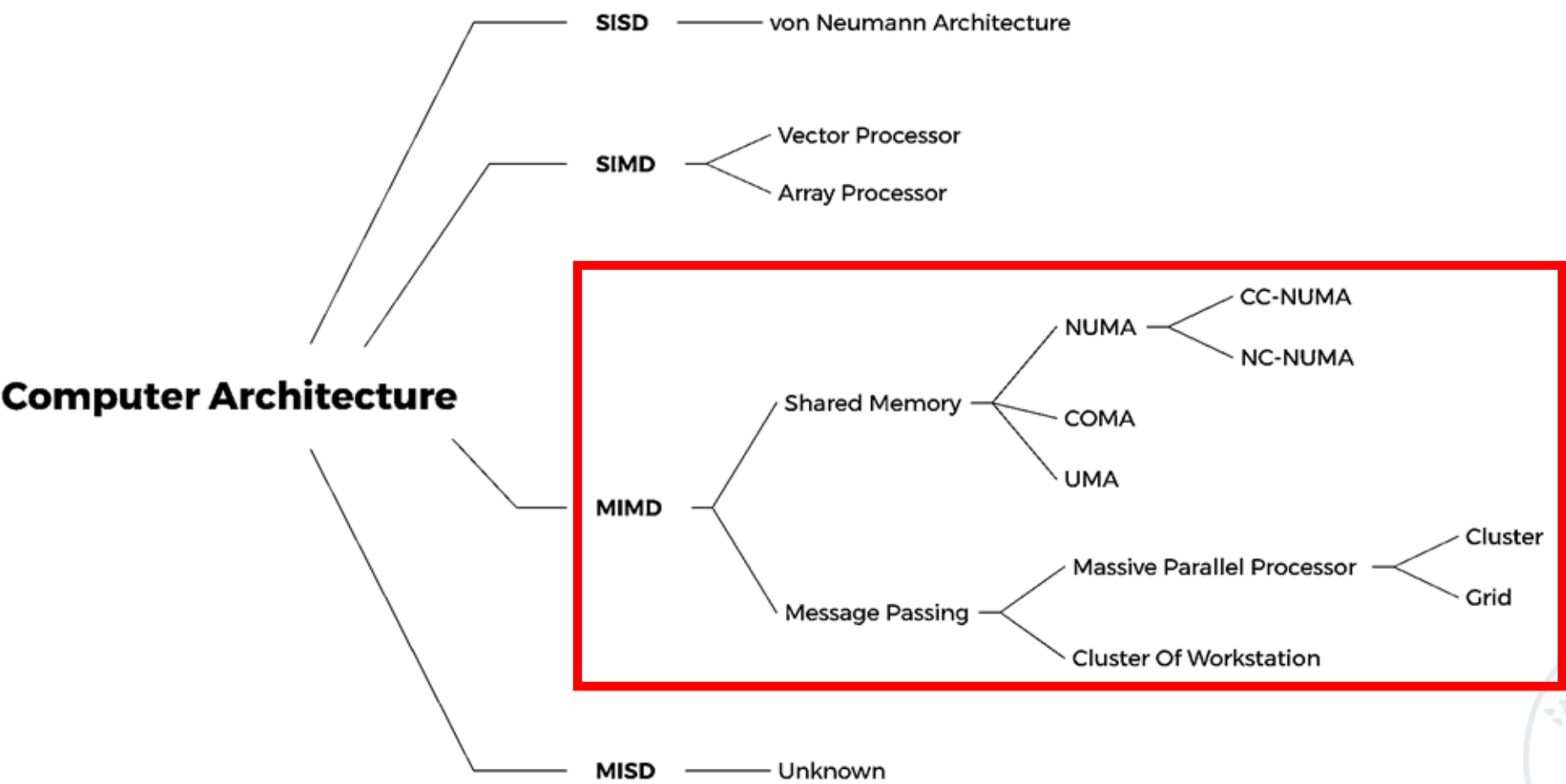


Multi-computer system based on message passing

- General multi-computer architecture
 - Each node is composed of one or more CPUs, RAMs, disks and other input/output devices and communication processors.
 - The communication processors are connected to each other through an interconnection network. A variety of different topologies, switching strategies and path finding algorithms can be used.



Flynn



MIMD Architecture

- Different memory access models of MIMD multiprocessor system
 - Uniform Memory Access (UMA)
 - Non Uniform Memory Access (NUMA)
 - Cache Only Memory Access (COMA)
- Further division of MIMD multi-computer system
 - Massively Parallel Processors (MPP)
 - Cluster of Workstations(COW)

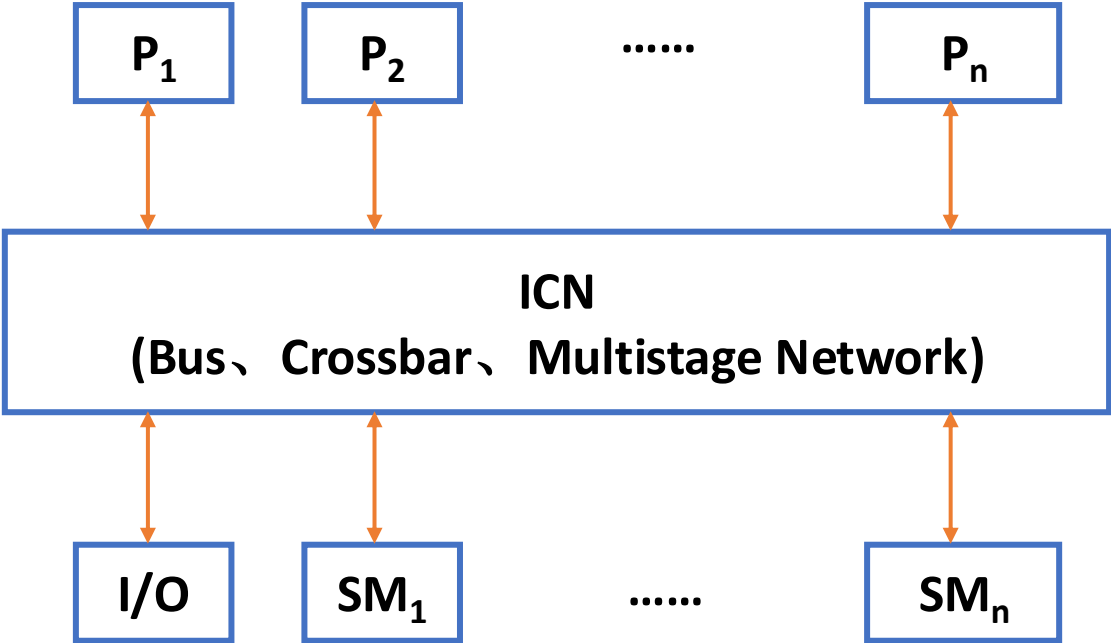


MIMD Architecture

- Different memory access models of MIMD multiprocessor system
 - Uniform Memory Access (UMA)
 - Non Uniform Memory Access (NUMA)
 - Cache Only Memory Access (COMA)
- Further division of MIMD multi-computer system
 - Massively Parallel Processors (MPP)
 - Cluster of Workstations(COW)

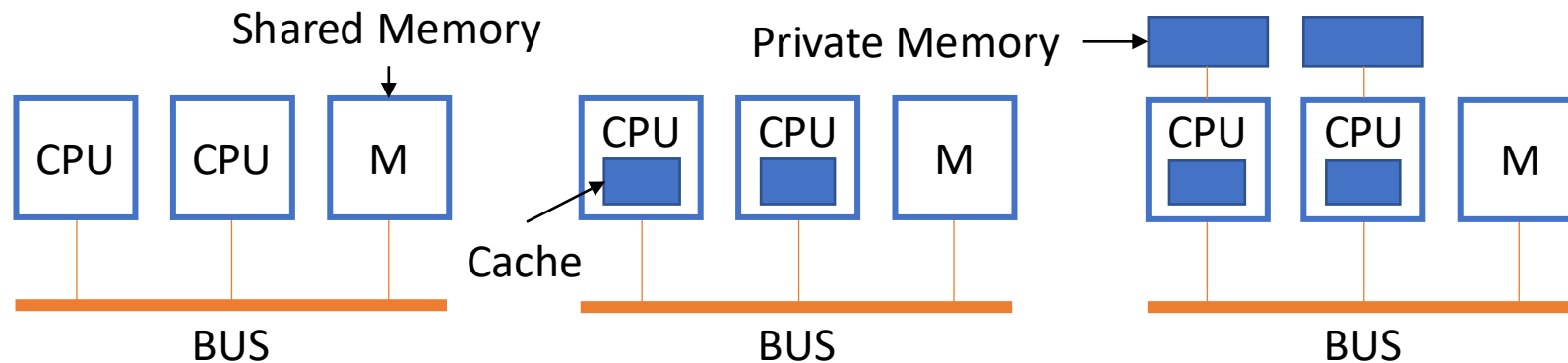


UMA



UMA Multi-processor system

- UMA system features
 - Physical memory is uniformly shared by all processors
 - It takes the same time for all processors to access any memory word
 - Each processor can be equipped with private cache or private memory
- UMA Multiprocessor System Based on Bus

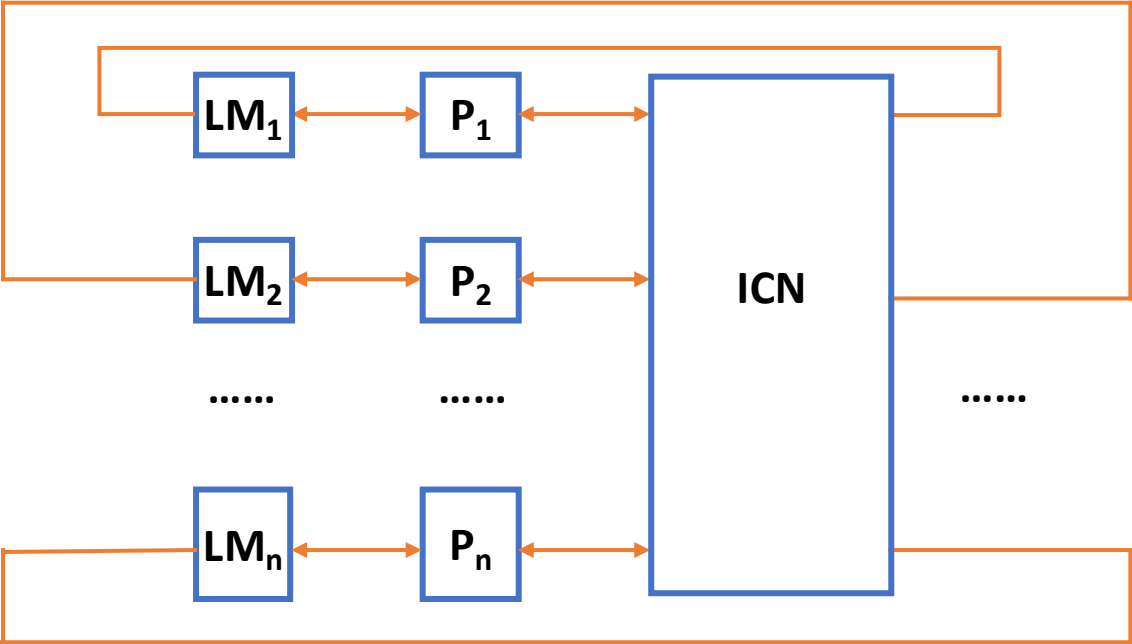


MIMD Architecture

- Different memory access models of MIMD multiprocessor system
 - Uniform Memory Access (UMA)
 - Non Uniform Memory Access (NUMA)
 - Cache Only Memory Access (COMA)
- Further division of MIMD multi-computer system
 - Massively Parallel Processors (MPP)
 - Cluster of Workstations(COW)



NUMA



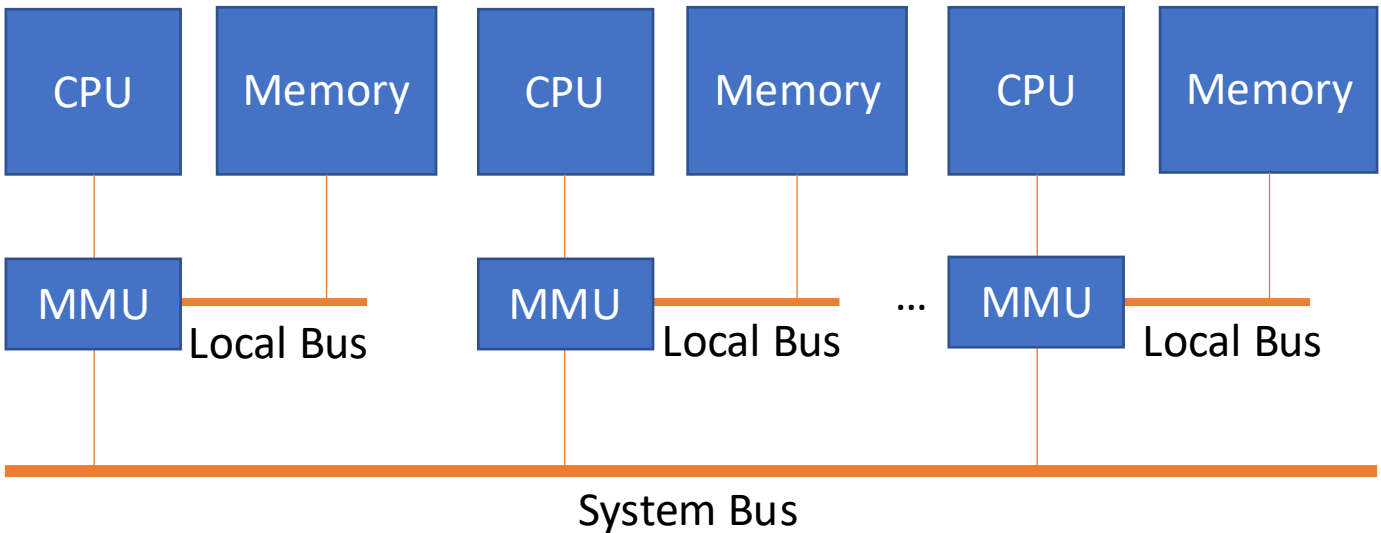
Shared local memory model LM: Local Memory P: Processor



NUMA

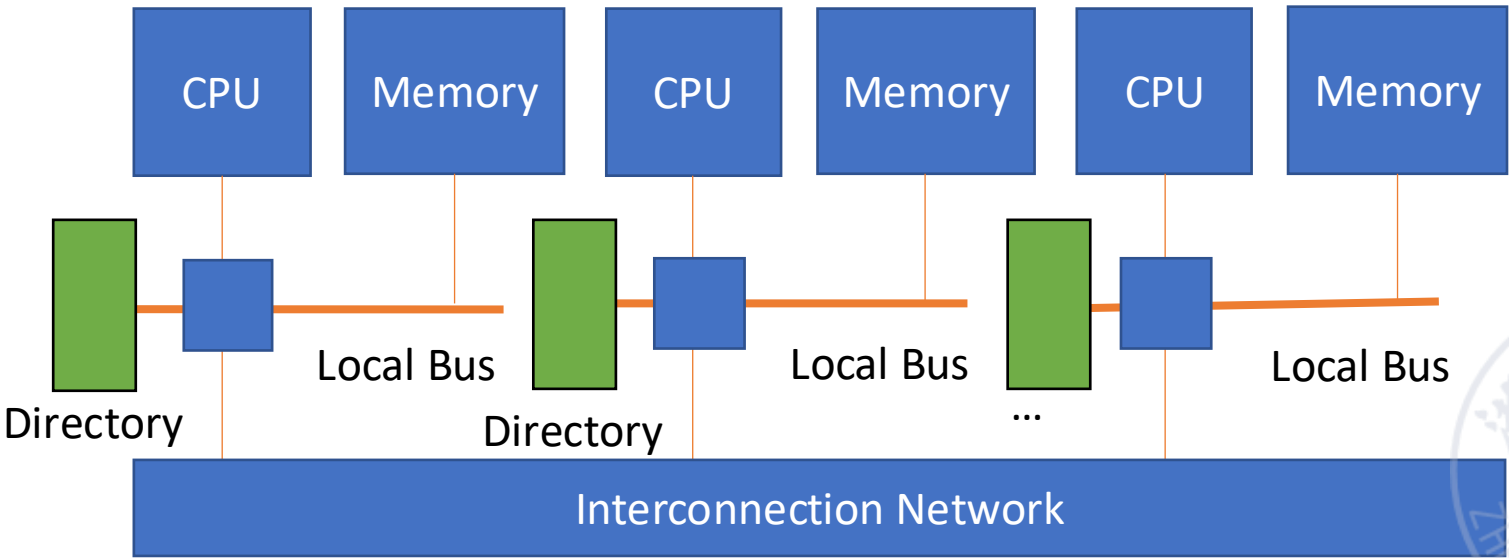
- NC-NUMA

Non Cache
Non-Uniform
Memory Access



- CC-NUMA

Coherent Cache
Non-Uniform
Memory Access



NUMA

- NUMA system features

- All CPUs share an **uniform address space**
- Use LOAD and STORE instructions to access remote memory
- Access to remote memory is slower than access to local memory
- The processor in the NUMA system can use **cache**

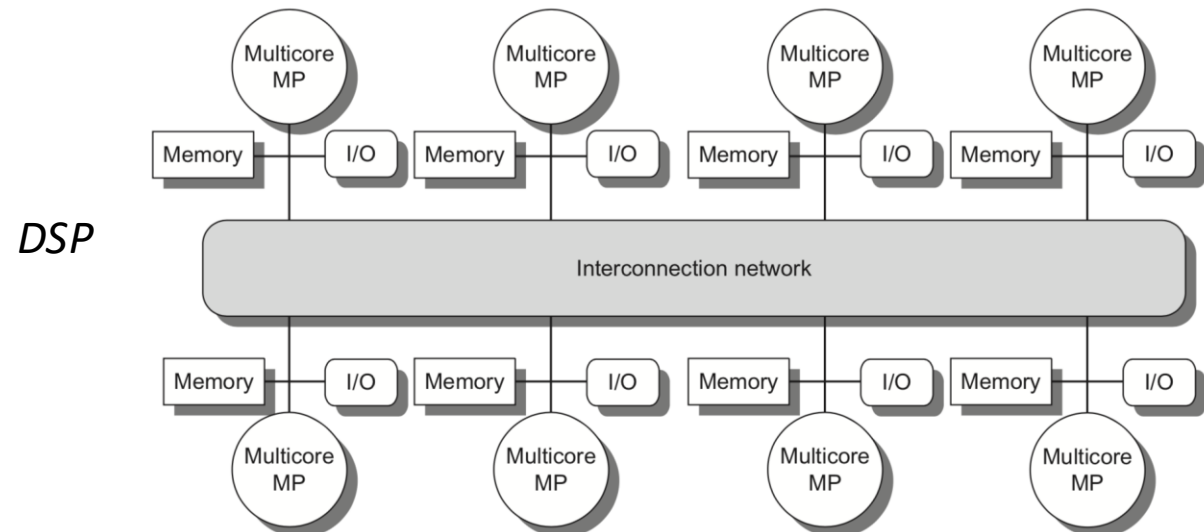
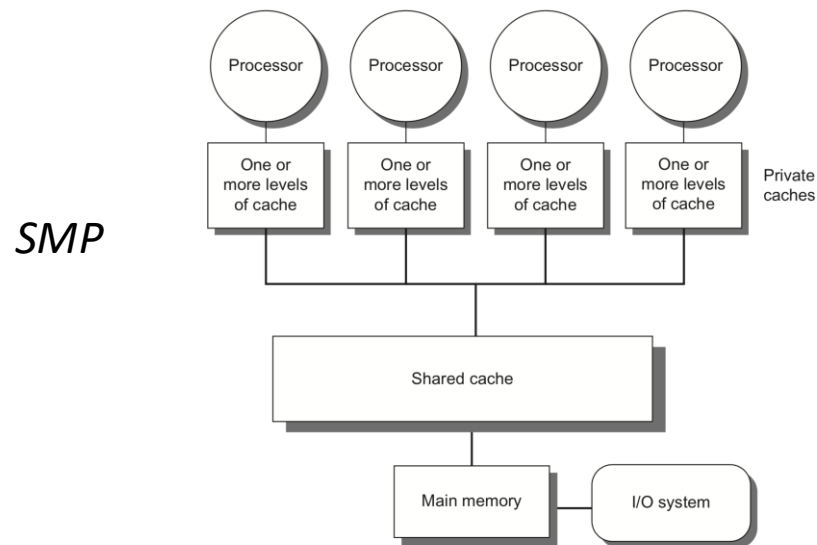
- NC-NUMA and CC-NUMA

- The NUMA system without Cache is called the NC-NUMA multiprocessor system, which means that the remote memory access time is not hidden in this system. If Cache is used, the system will be called a CC-NUMA multiprocessor system.



UMA and NUMA

- UMA is also called *symmetric (shared-memory) multiprocessors (SMP)* or *centralized shared-memory multiprocessors*.
- NUMA is called *distributed shared-memory multiprocessor (DSP)*.

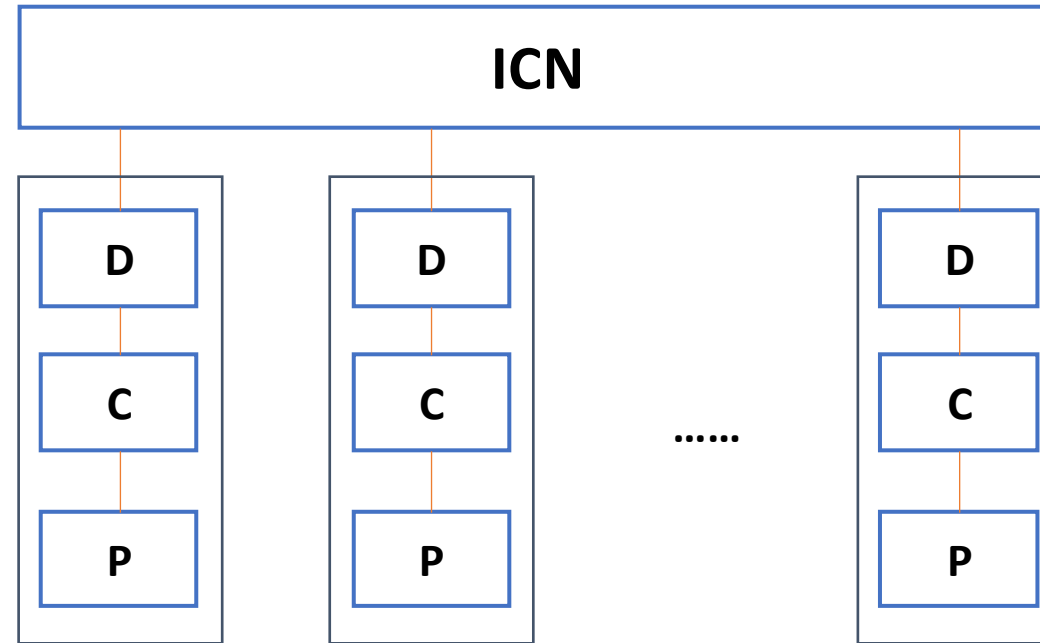


MIMD Architecture

- Different memory access models of MIMD multiprocessor system
 - Uniform Memory Access ([UMA](#))
 - Non Uniform Memory Access ([NUMA](#))
 - Cache Only Memory Access ([COMA](#))
- Further division of MIMD multi-computer system
 - Massively Parallel Processors (MPP)
 - Cluster of Workstations(COW)



COMA



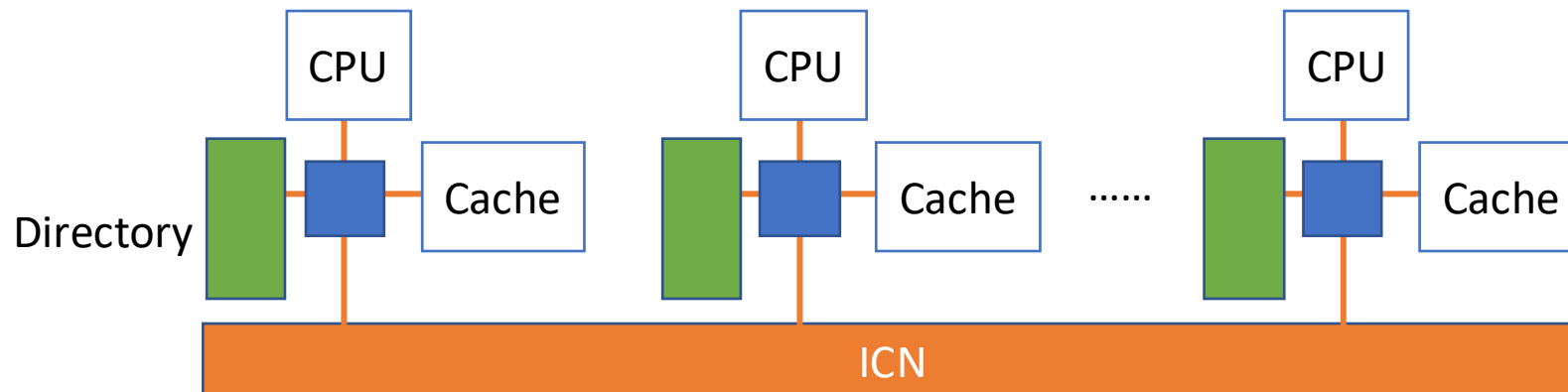
D: Cache Directory **C:** Cache **P:** Processor



COMA

Characteristics of COMA

- COMA is a special case of NUMA. There is no storage hierarchy in each processor node, and all caches form a uniform address space.
- Use the distributed cache directory for remote cache access. When using COMA, the data can be allocated arbitrarily at the beginning, because it will eventually be moved to the place where it is used at runtime.



MIMD Architecture

- Different memory access models of MIMD multiprocessor system
 - Uniform Memory Access (UMA)
 - Non Uniform Memory Access (NUMA)
 - Cache Only Memory Access (COMA)
- Further division of MIMD multi-computer system
 - Massively Parallel Processors (MPP)
 - Cluster of Workstations(COW)



MPP

- The MPP system is a massively parallel computer system composed of hundreds of processors.
- In the past, it was mainly used for calculation-oriented occasions such as scientific calculation and engineering simulation, but it is also widely used in commercial and network applications.
- Development is difficult, the price is high, and the market is limited. It is a symbol of the country's comprehensive strength.



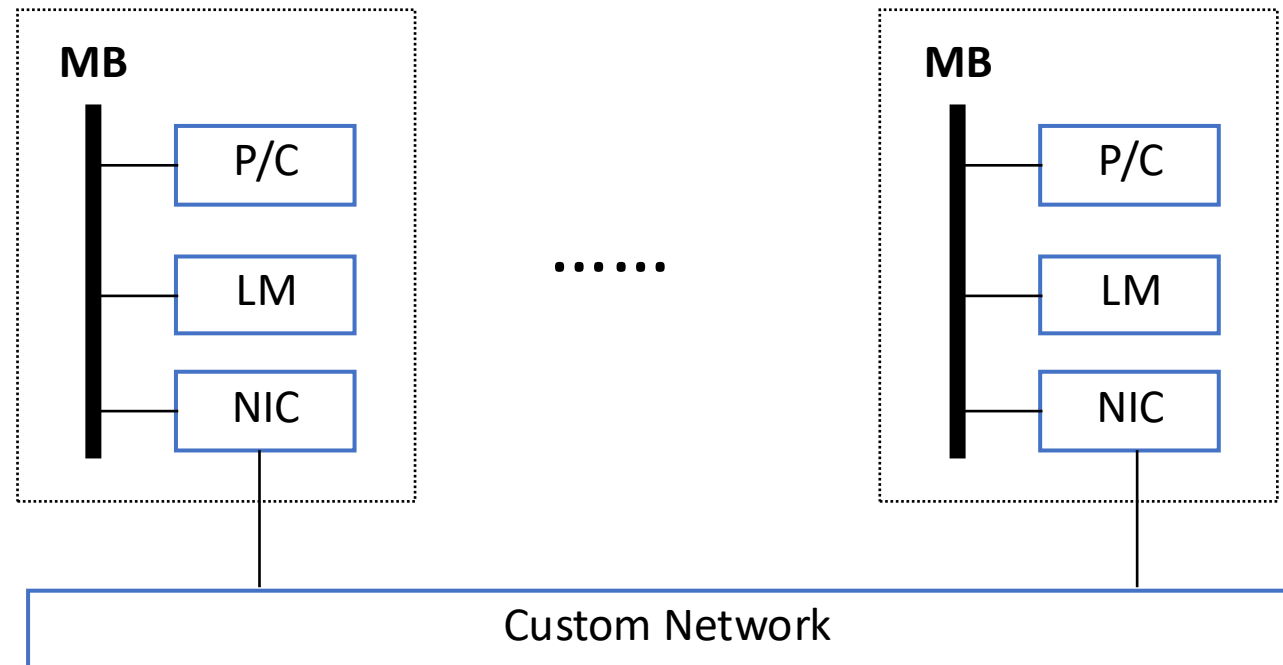
Characteristics of MPP

- MPP systems generally use standard commercial CPUs as their processors.
- The MPP system uses a high-performance private interconnection network, which can deliver messages with low latency and high bandwidth.
- The MPP system has powerful input/output capabilities.
- The MPP system is capable of special fault-tolerant processing.



MPP

LM: Local Memory
NIC: Network interface circuit
MB: Memory Bus



MIMD Architecture

- Different memory access models of MIMD multiprocessor system
 - Uniform Memory Access (UMA)
 - Non Uniform Memory Access (NUMA)
 - Cache Only Memory Access (COMA)
- Further division of MIMD multi-computer system
 - Massively Parallel Processors (MPP)
 - Cluster of Workstations(COW)

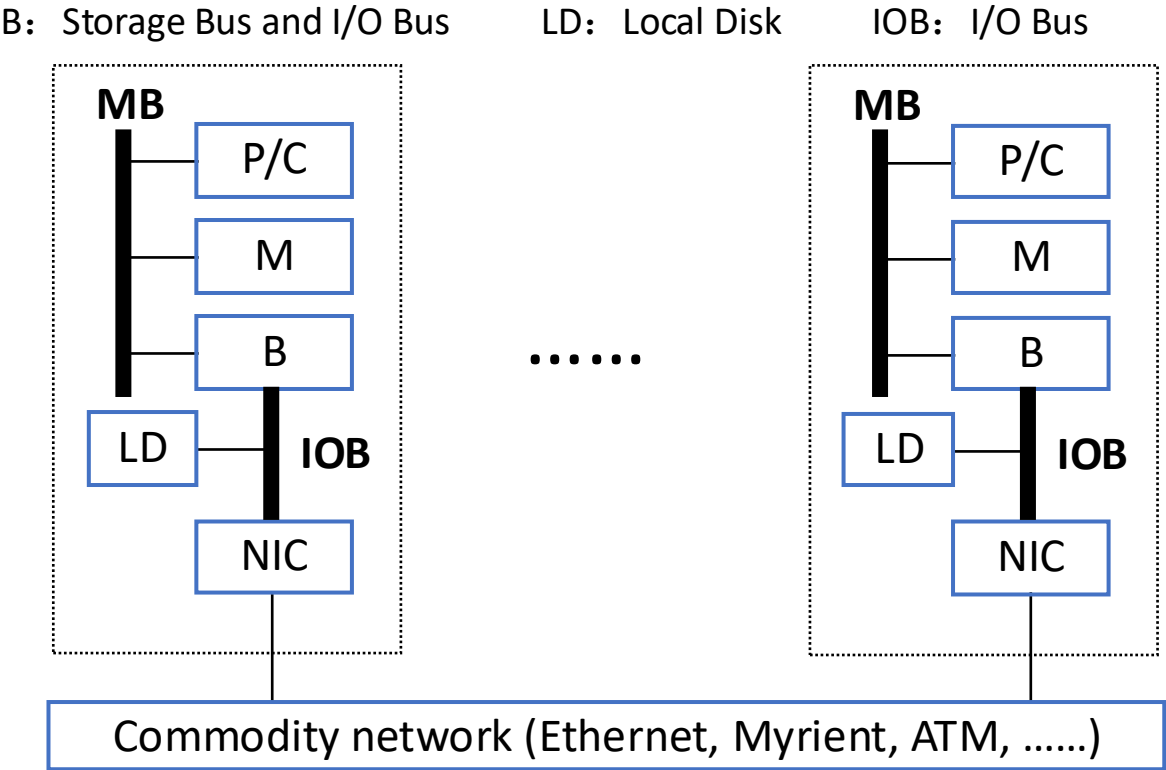


COW

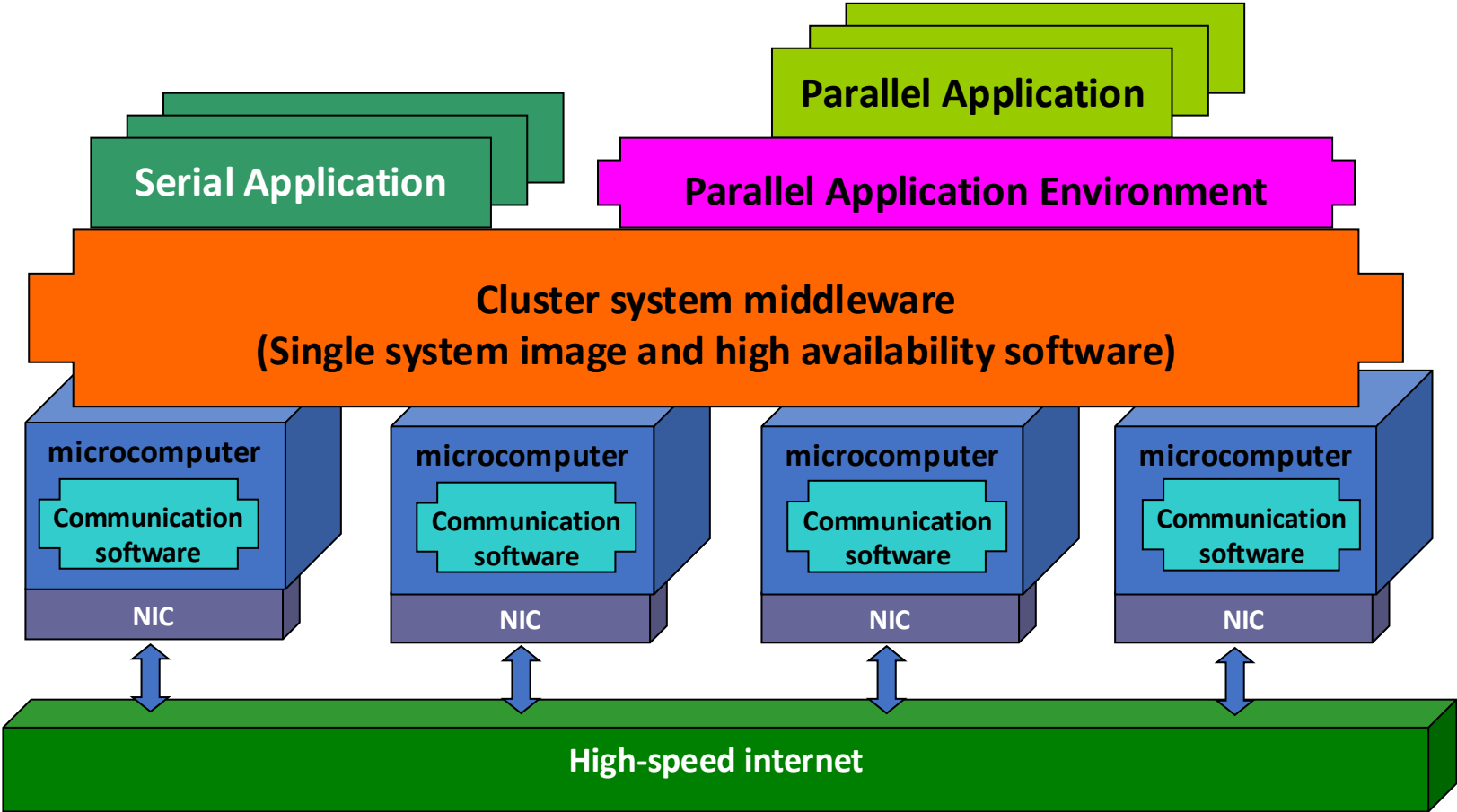
- The COW system is composed of a large number of PCs or workstations connected together through a commercial network.
- COW can be assembled completely using commercially available components. These commercial components are mass-produced products, so they can achieve higher cost performance.
- There are two main types of COW that dominate: centralized and decentralized.



COW



COW Architecture



Challenges of Parallel Processing

The application of multiprocessors ranges from running independent tasks with essentially no communication to running parallel programs where threads must communicate to complete the task.

The first hurdle has to do with the limited parallelism available in programs.

The second arises from the relatively high cost of communications.

Both can be explained with Amdahl's Law.



Challenges of Parallel Processing

Example:

Suppose you want to achieve a speedup of 80 with 100 processors. What fraction of the original computation can be sequential?

$$\text{Speedup}_{\text{overall}} = \frac{\text{Execution time}_{\text{old}}}{\text{Execution time}_{\text{new}}} = \frac{1}{(1 - \text{Fraction}_{\text{enhanced}}) + \frac{\text{Fraction}_{\text{enhanced}}}{\text{Speedup}_{\text{enhanced}}}}$$

$$80 = \frac{1}{\frac{\text{Fraction}_{\text{parallel}}}{100} + (1 - \text{Fraction}_{\text{parallel}})}$$

$$\text{Fraction}_{\text{parallel}} = \frac{80 - 1}{79.2}$$
$$\text{Fraction}_{\text{parallel}} = 0.9975$$

Thus, to achieve a speedup of 80 with 100 processors, only 0.25% of the original computation can be sequential!



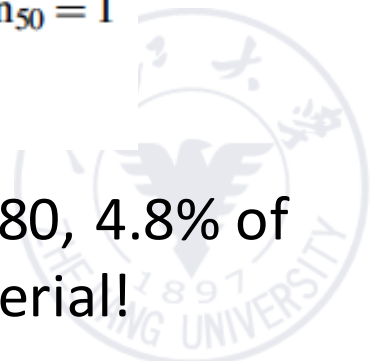
Challenges of Parallel Processing

Example:

Suppose we have an application running on a 100-processor multiprocessor, and assume that application can use 1, 50, or 100 processors. If we assume that 95% of the time we can use all 100 processors, how much of the remaining 5% of the execution time must employ 50 processors if we want a speedup of 80?

$$\text{Speedup} = \frac{1}{\frac{\text{Fraction}_{100}}{\text{Speedup}_{100}} + \frac{\text{Fraction}_{50}}{\text{Speedup}_{50}} + (1 - \text{Fraction}_{100} - \text{Fraction}_{50})}$$
$$80 = \frac{1}{\frac{0.95}{100} + \frac{\text{Fraction}_{50}}{50} + (1 - 0.95 - \text{Fraction}_{80})}$$
$$\begin{aligned} 0.76 + 1.6 \times \text{Fraction}_{50} + 4.0 - 80 \times \text{Fraction}_{50} &= 1 \\ 4.76 - 78.4 \times \text{Fraction}_{50} &= 1 \\ \text{Fraction}_{50} &= 0.048 \end{aligned}$$

If 95% of an application can use 100 processors perfectly, to get a speedup of 80, 4.8% of the remaining time must be spent using 50 processors and only 0.2% can be serial!



Challenges of Parallel Processing

Example:

Suppose we have an application running on a 32-processor multiprocessor that has a 100 ns delay to handle a reference to a remote memory. For this application, assume that all the references except those involving communication hit in the local memory hierarchy. Processors are stalled on a remote request, and the processor clock rate is 4 GHz. If the base CPI (assuming that all references hit in the cache) is 0.5, and 0.2% of the instructions involve a remote communication reference, how much faster is the multiprocessor with no communication?

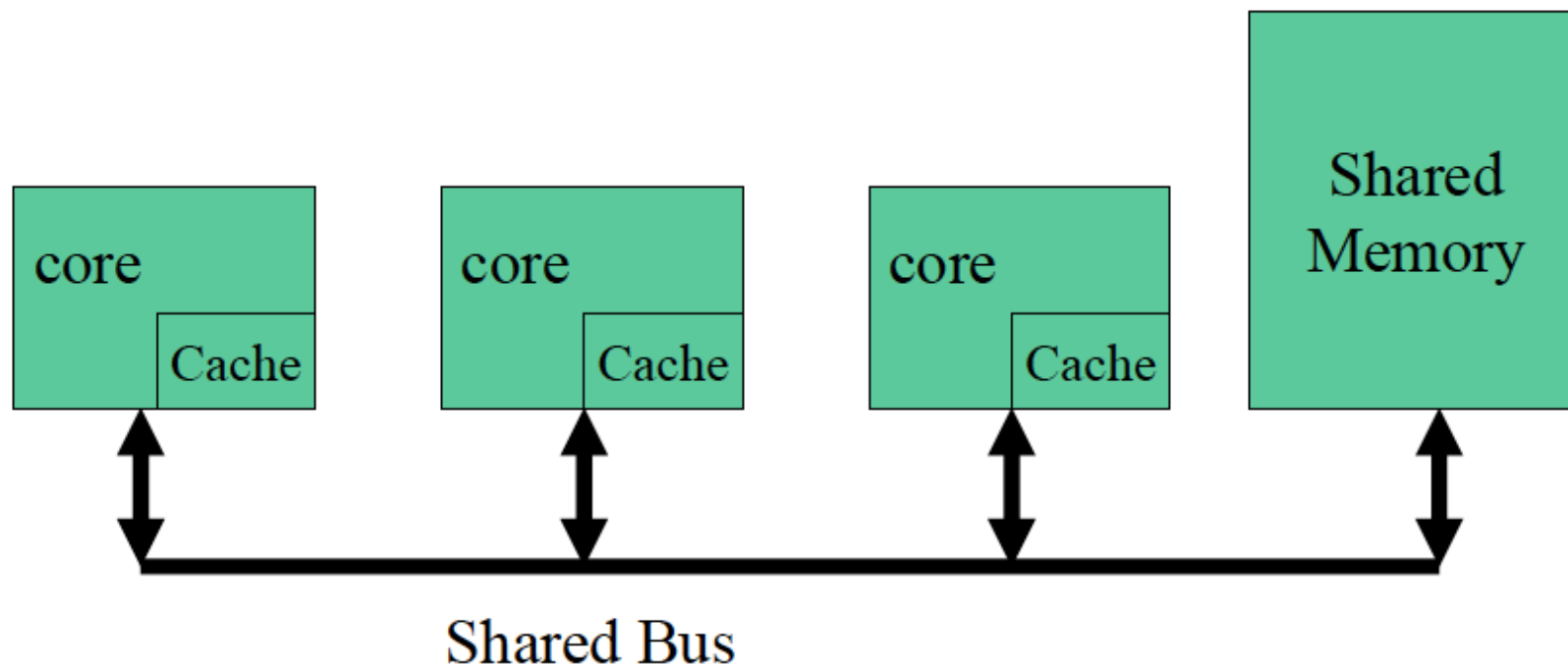
$$CPI = Base\ CPI + Remote\ request\ rate \times Remote\ request\ cost = 0.5 + 0.2\% \times Remote\ request\ cost$$

$$The\ remote\ request\ cost\ is: \quad Remote\ access\ cost / Cycle\ time = 100ns / 0.25ns = 400\ cycles$$

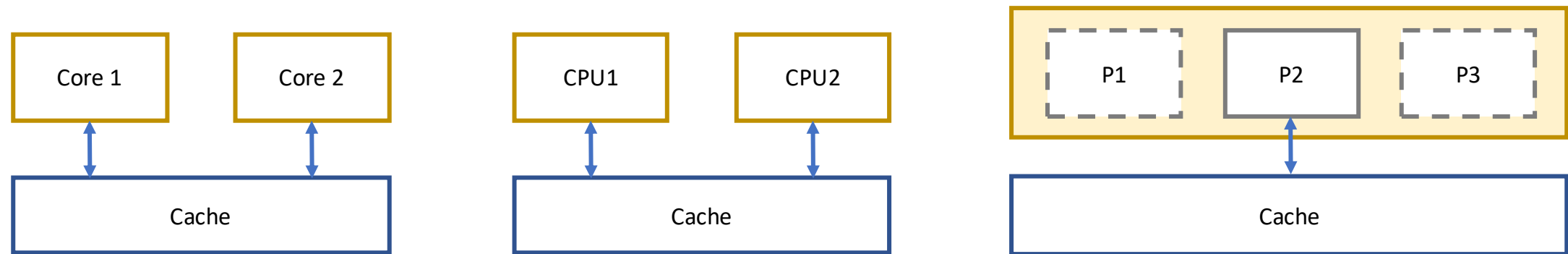
$$SP = CPI / Base\ CPI = (0.5 + 0.20\% \times 400) / 0.5 = 1.3 / 0.5 = 2.6$$



Challenges of Multi-process system based on Shared Memory



Challenges of Multi-process system based on Shared Memory

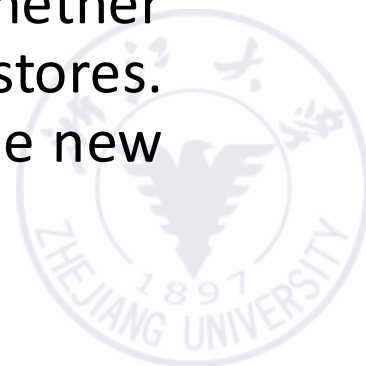


Time step	Event	Cache contents for CPU A	Cache contents for CPU B	Memory contents for location X
0				0
1	CPU A reads X	0		0
2	CPU B reads X	0	0	0
3	CPU A stores 1 into X	1	0	1



Memory Consistency and Cache Coherence

- Memory Consistency **Need Memory Consistency Model**
 - When a written value will be returned by a read
 - If a processor writes location A followed by location B, any processor that sees the new value of B must also see the new value of A
- Cache Coherence **Need Cache Coherence Protocol**
 - All reads by any processor must return the most recently written value
 - Writes to the same location by any two processors are seen in the same order by all processors
 - Correct coherence ensures that a programmer cannot determine whether and where a system has caches by analyzing the results of loads and stores. This is because correct coherence ensures that the caches never enable new or different *functional* behavior.



Cache Coherence

■ *Migration*: A data item can be moved to a local cache and used there in a transparent fashion. Migration reduces both the latency to access a shared data item that is allocated remotely and the bandwidth demand on the shared memory.

■ *Replication*: When shared data are being simultaneously read, the caches make a copy of the data item in the local cache. Replication reduces both latency of access and contention for a read shared data item.



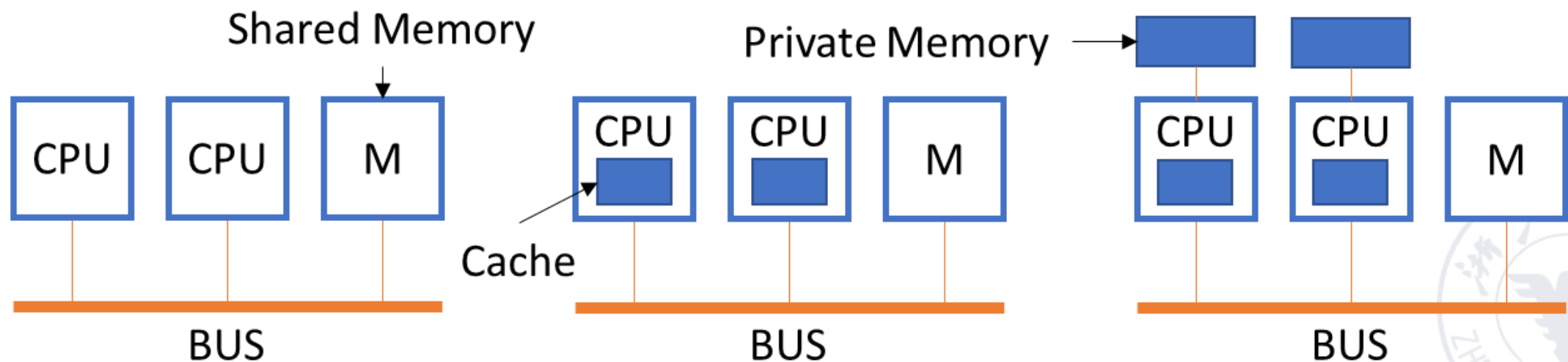
Cache Coherence

- Causes of Cache coherence problems
 - In modern parallel computers, processors often have Cache. Memory data may have multiple copies in the entire system. This leads to the [cache coherence problem](#).
- Cache coherence protocol
 - A set of rules implemented by Cache, CPU, and memory to prevent different versions of the same data from appearing in multiple Caches forms a [cache coherence protocol](#).
 - [Bus snooping protocol](#)
 - [Directory based protocol](#)



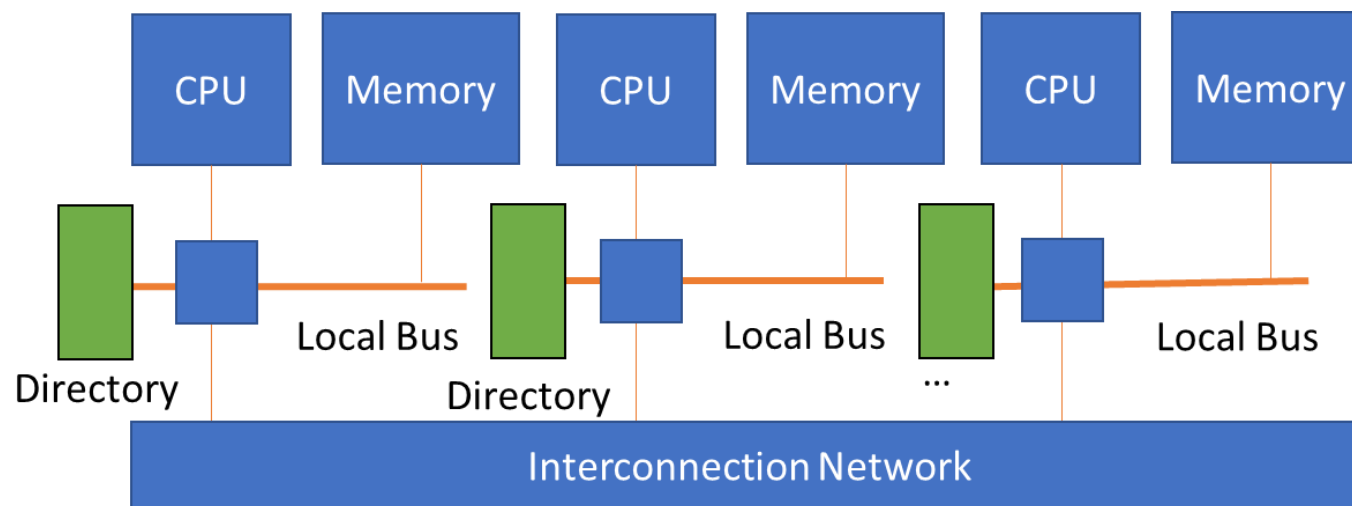
Cache Coherence

- For UMA: *Snoopy coherence protocols*
 - In the snoopy coherence protocols, all processors snoop the bus. When a processor modifies the data in the private cache, it broadcasts invalid information or updated data on the bus to invalidate or update other copies.

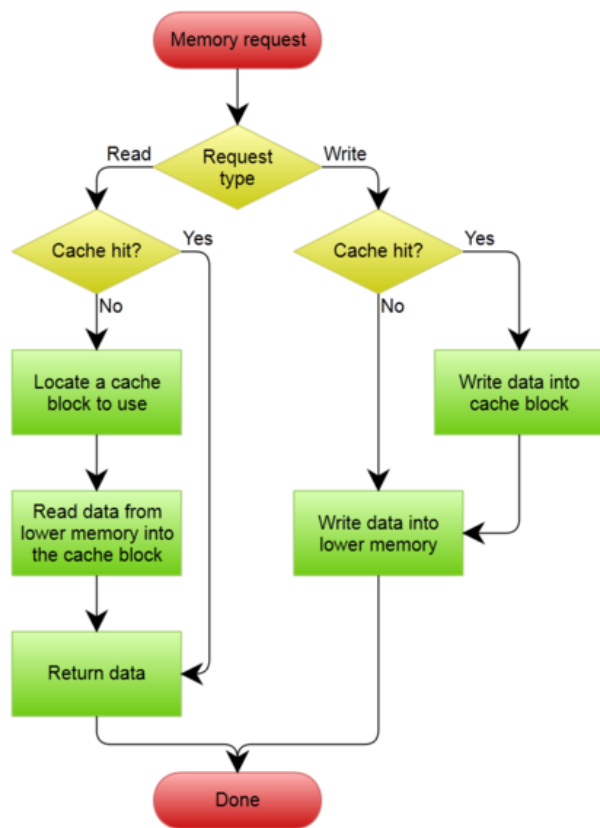


Cache Coherence

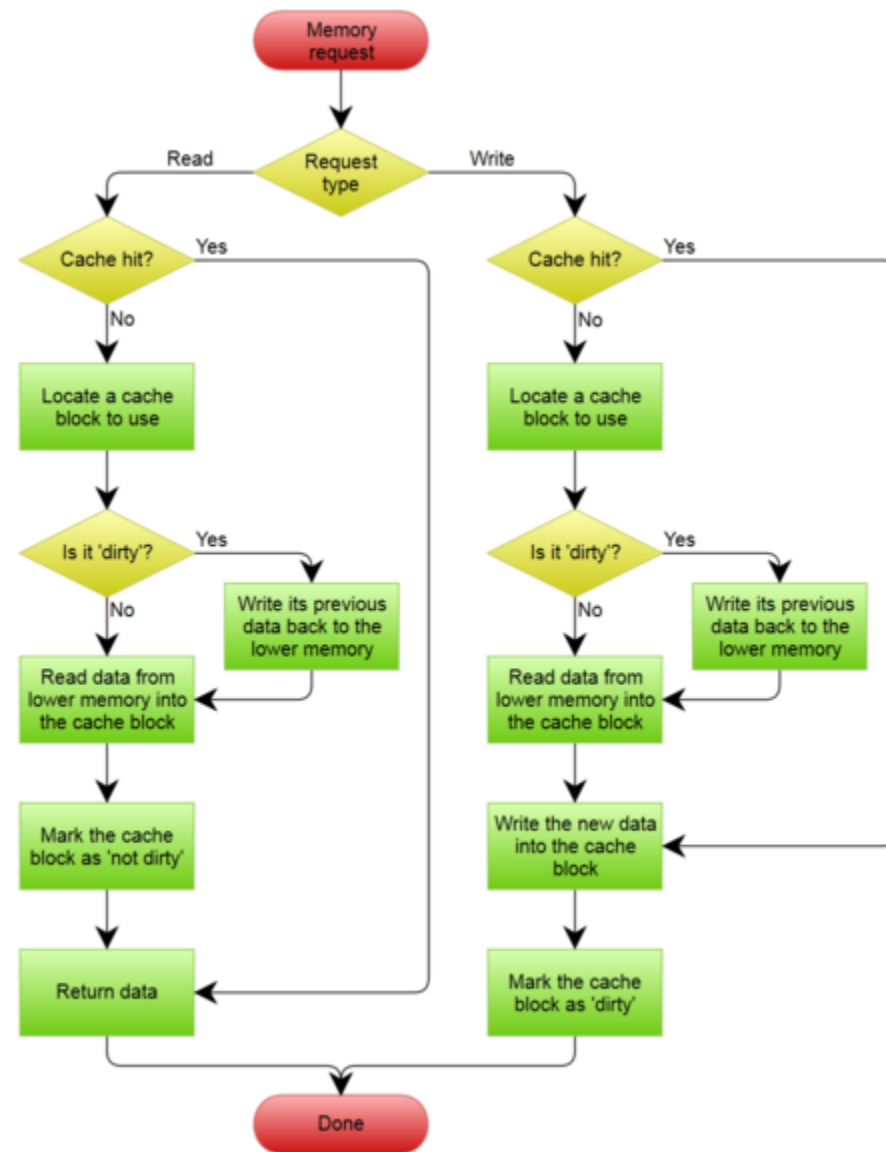
- For NUMA: *Directory protocol*
 - The directory protocol uses a directory to record which processors in the system have copies of certain storage blocks in the cache. When a processor wants to write a shared block, it sends an invalid signal to those processors that have copies of the block through the directory in a "point-to-point" way, so that all other copies are invalidated.



Cache Coherence



A Write-Through cache with No-Write Allocation



A Write-Back cache with Write Allocation

Snoopy Coherence Protocols

- Write-through
 - While writing the data in the cache line, the content in the corresponding memory is also modified, and the data in the memory is kept up to date at any time.
- Write-back
 - The write operation does not directly write to the memory. On the contrary, when the cache line is modified, a certain bit in the cache is set to indicate that the data in the cache line is correct but the data in the memory is out of date. Of course, the line will eventually be written back to memory, but it may be after multiple write operations.



Write-through Cache Coherency Protocol

- Four situations when the monitoring cache performs read and write operations according to this protocol

	Local Request	Remote Request
Read Miss	Access data from memory	
Read Hit	Use local cache data	
Write Miss	Modify data in memory	
Write Hit	Modify cache and memory	Invalidate the cache item

- There are many changes in the basic protocol of write direct Cache consistency
 - Whether to use **Update Strategy** or **Invalidate Strategy** for remote write hits
 - Whether to transfer the corresponding word into the cache when the cache write is missing, this is whether to use the **Write-allocate Policy**.



Snoopy Coherence Protocols

- Write invalidate protocol

It invalidates other copies on a write.

Processor activity	Bus activity	Contents of processor A's cache	Contents of processor B's cache	Contents of memory location X
				0
Processor A reads X	Cache miss for X	0		0
Processor B reads X	Cache miss for X	0	0	0
Processor A writes a 1 to X	Invalidation for X	1		0
Processor B reads X	Cache miss for X	1	1	1

An invalidation protocol works on a snooping bus for a single cache block (X) with write-back caches.



Snoopy Coherence Protocols

- Write invalidate protocol

It invalidates other copies on a write.

- Write update / write broadcast protocol

Update all the cached copies of a data item when that item is written.



Snoopy Coherence Protocols

- Write invalidate protocol

Implementation

three block states (**MSI protocol**)

- **Invalid**

- **Shared**

indicates that the block in the private cache is potentially shared

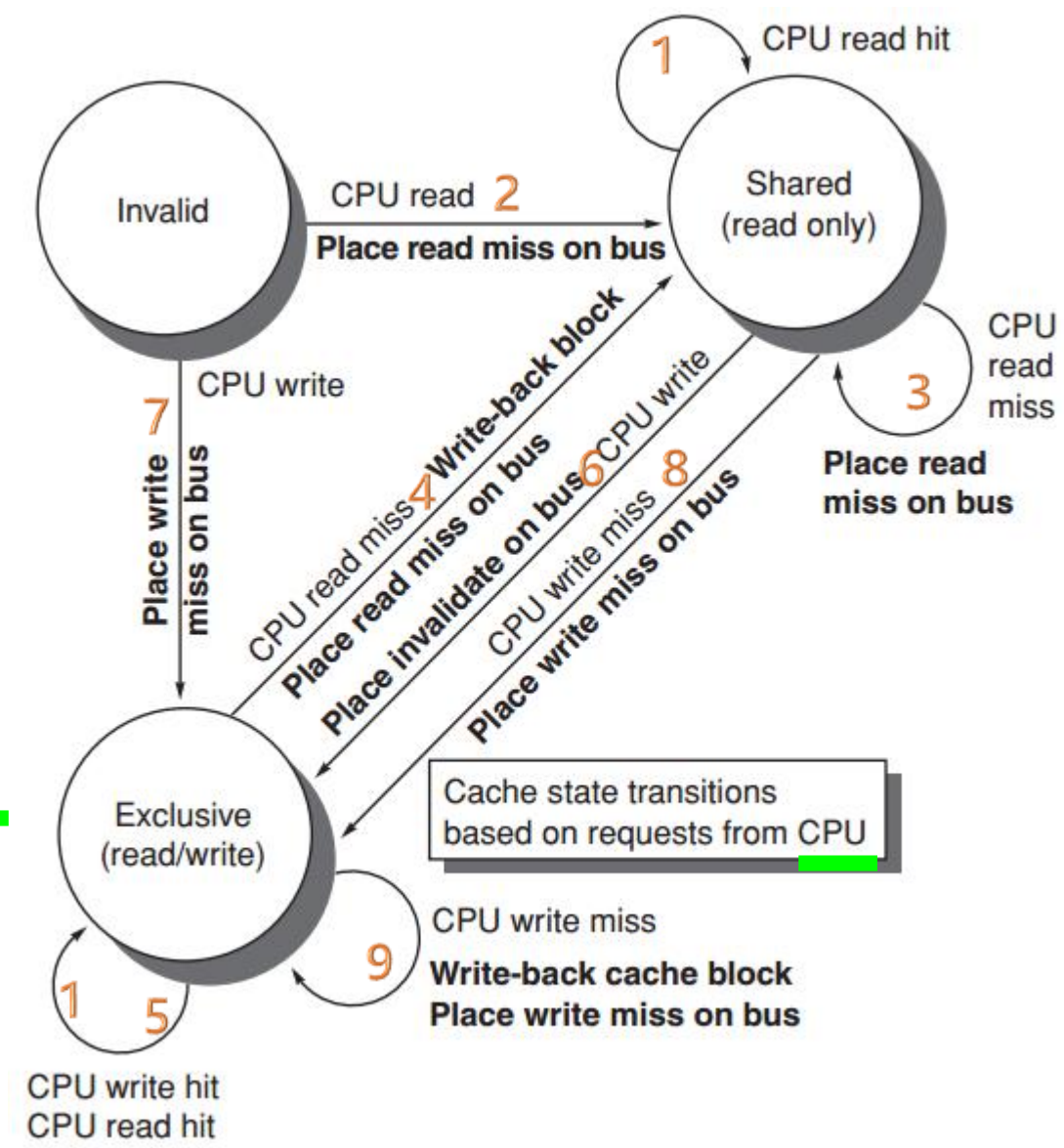
- **Modified**

indicates that the block has been updated in the private cache;
implies that the block is **exclusive**



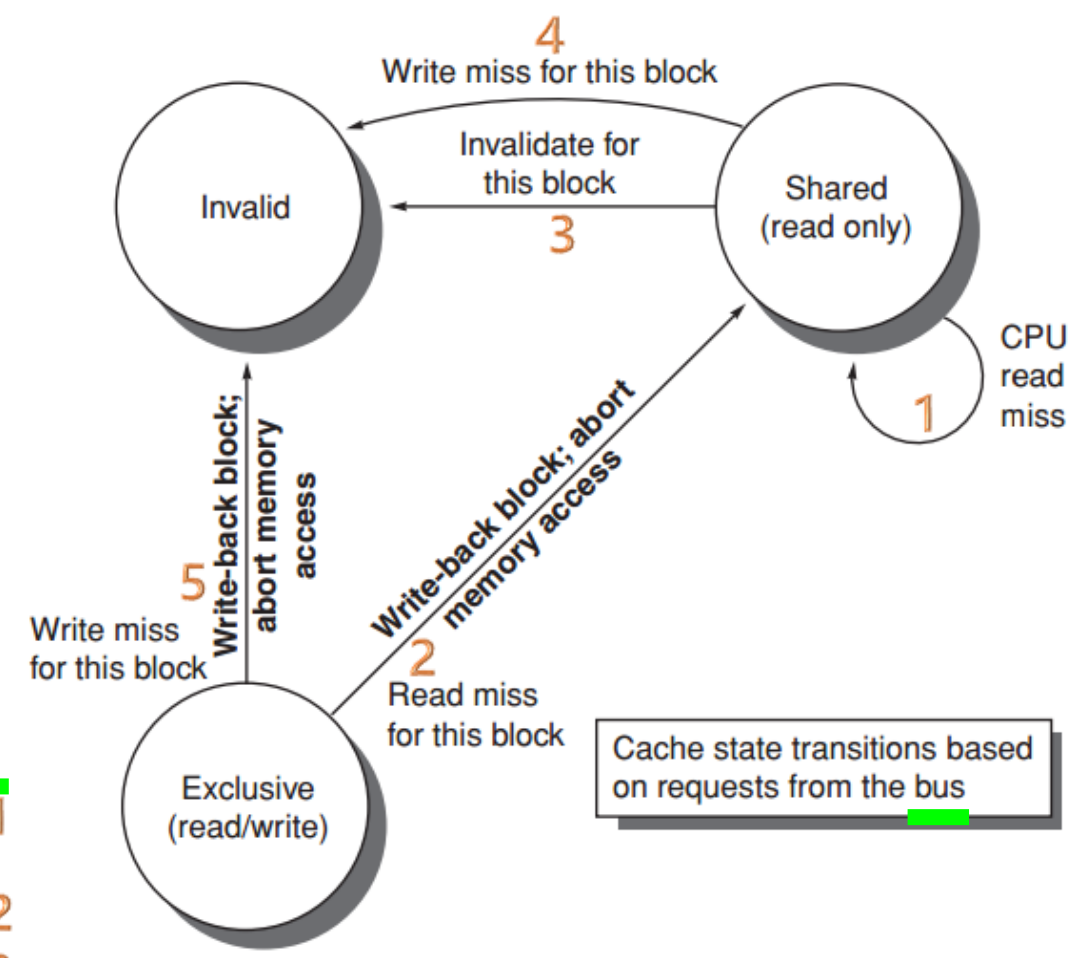
Write Invalidation Protocol (write back)

Request	Source	State of addressed cache block	Type of cache action	Function and explanation	
Read hit	processor	shared or modified	normal hit	Read data in cache.	1
Read miss	processor	invalid	normal miss	Place read miss on bus.	2
Read miss	processor	shared	replacement	Address conflict miss: place read miss on bus.	3
Read miss	processor	modified	replacement	Address conflict miss: write back block, then place read miss on bus.	4
Write hit	processor	modified	normal hit	Write data in cache.	5
Write hit	processor	shared	coherence	Place invalidate on bus. These operations are often called upgrade or <i>ownership</i> misses, since they do not fetch the data but only change the state.	6
Write miss	processor	invalid	normal miss	Place write miss on bus.	7
Write miss	processor	shared	replacement	Address conflict miss: place write miss on bus.	8
Write miss	processor	modified	replacement	Address conflict miss: write back block, then place write miss on bus.	9
Read miss	bus	shared	no action	Allow memory to service read miss.	
Read miss	bus	modified	coherence	Attempt to share data: place cache block on bus and change state to shared.	
Invalidate	bus	shared	coherence	Attempt to write shared block; invalidate the block.	
Write miss	bus	shared	coherence	Attempt to write block that is shared; invalidate the cache block.	
Write miss	bus	modified	coherence	Attempt to write block that is exclusive elsewhere: write back the cache block and make its state invalid.	

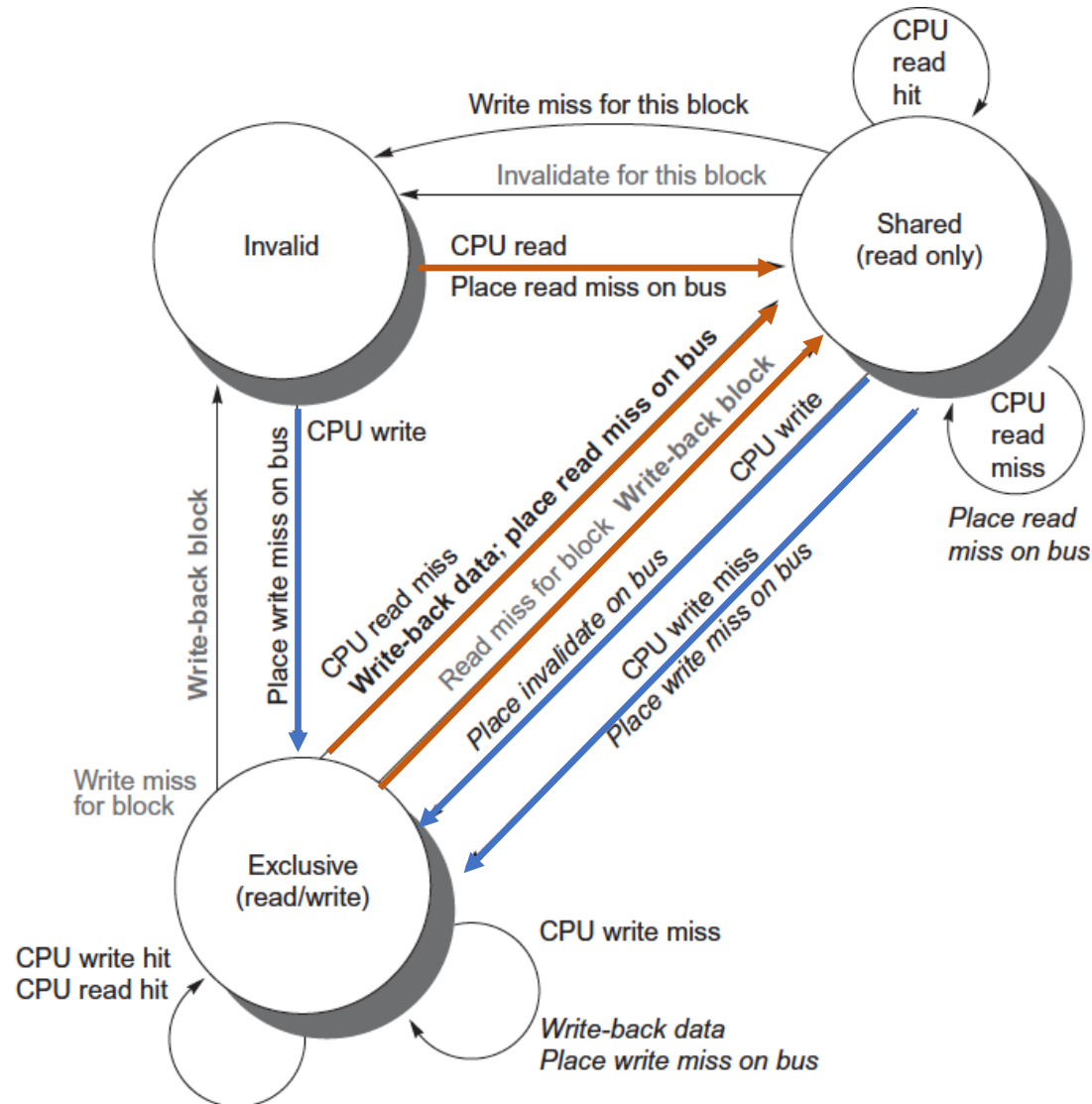


Write Invalidation Protocol (write back)

Request	Source	State of addressed cache block	Type of cache action	Function and explanation
Read hit	processor	shared or modified	normal hit	Read data in cache.
Read miss	processor	invalid	normal miss	Place read miss on bus.
Read miss	processor	shared	replacement	Address conflict miss: place read miss on bus.
Read miss	processor	modified	replacement	Address conflict miss: write back block, then place read miss on bus.
Write hit	processor	modified	normal hit	Write data in cache.
Write hit	processor	shared	coherence	Place invalidate on bus. These operations are often called upgrade or <i>ownership</i> misses, since they do not fetch the data but only change the state.
Write miss	processor	invalid	normal miss	Place write miss on bus.
Write miss	processor	shared	replacement	Address conflict miss: place write miss on bus.
Write miss	processor	modified	replacement	Address conflict miss: write back block, then place write miss on bus.
Read miss	bus	shared	no action	Allow memory to service read miss.
Read miss	bus	modified	coherence	Attempt to share data: place cache block on bus and change state to shared.
Invalidate	bus	shared	coherence	Attempt to write shared block; invalidate the block.
Write miss	bus	shared	coherence	Attempt to write block that is shared; invalidate the cache block.
Write miss	bus	modified	coherence	Attempt to write block that is exclusive elsewhere: write back the cache block and make its state invalid.



Write Invalidation Protocol (write back)



MSI protocol

The initial cache and memory state for a shared memory multi-processor system are shown in the following figure. Each core processor has a **4-line direct-mapped write back** cache. Basic **write invalidation snooping protocol** is used. Please show the action results (ie. updated parts in the caches and memory) for the three action as that in the example, assuming that all the actions are applied to the initial cache and memory states in the figure. In cache state, I means Invalid, S means Shared, M means Modified.

Action specification format:

C#, R, <address Axxx> represents

Core # reads address Axxx.

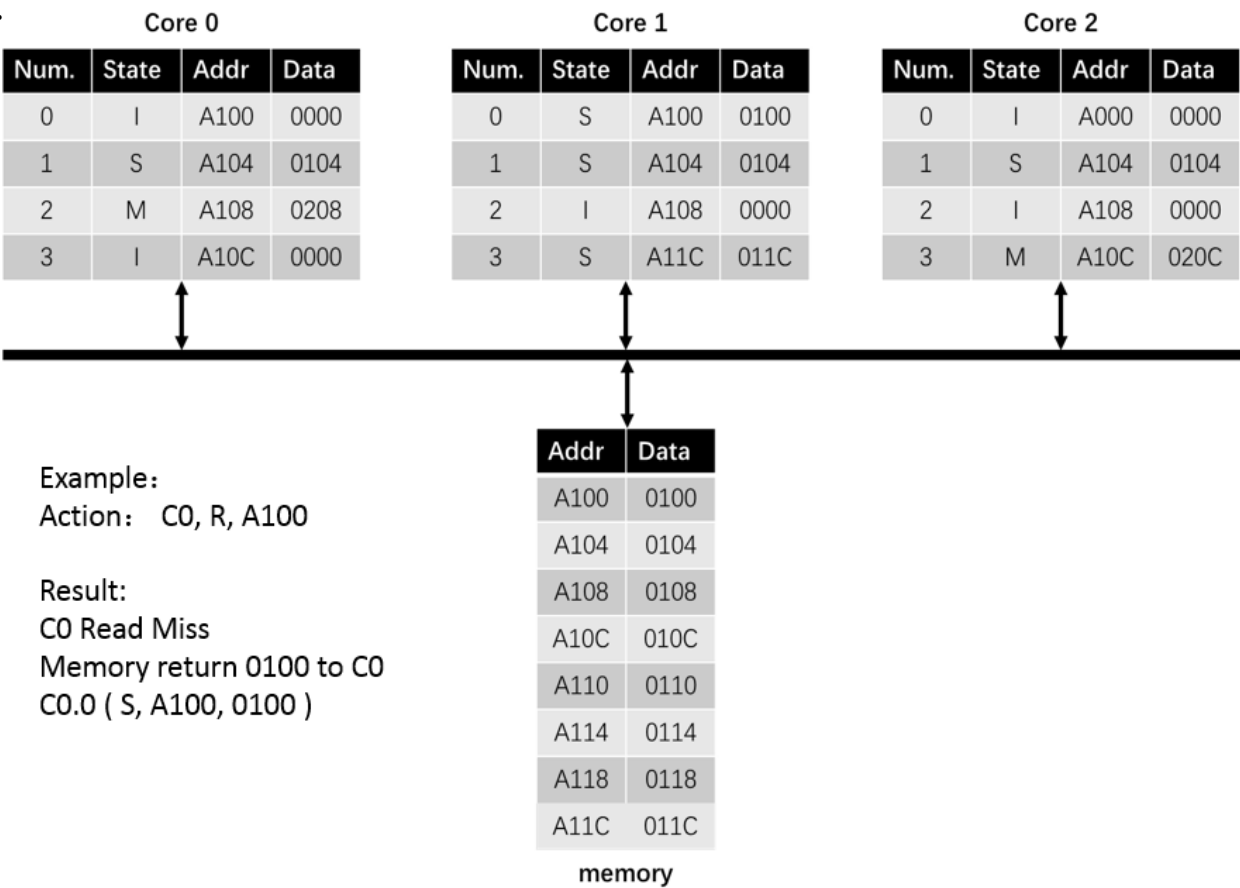
C#, W, <address Axxx>, <value V> represents

Core # writes value V into address Axxx.

Cx.y represents cache line y in core x.

Memory A100, 0000 → 0100 represents the value in memory location A100 is updated from 0000 to 0100.

- (1) Action: C0, R, A10C
- (2) Action: C1, W, A104, 0204
- (3) Action: C0, W, A118, 0308



MSI protocol

The initial cache and memory state for a shared memory multi-processor system are shown in the following figure. Each core processor has a **4-line direct-mapped write back** cache. Basic **write invalidation snooping protocol** is used. Please show the action results (ie. updated parts in the caches and memory) for the three action as that in the example, assuming that all the actions are applied to the initial cache and memory states in the figure. In cache state, I means Invalid, S means Shared, M means Modified.

Action specification format:

C#, R, <address Axxx> represents

Core # reads address Axxx.

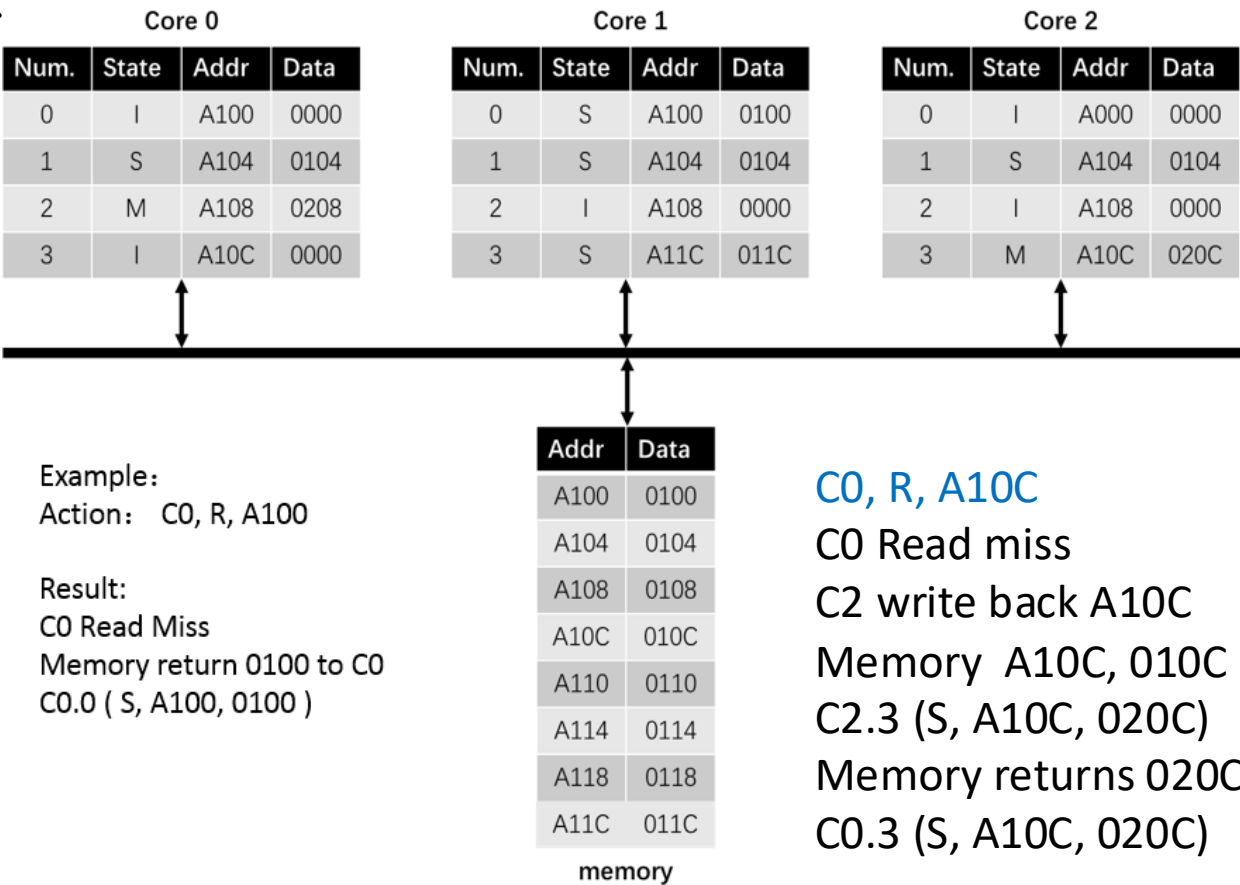
C#, W, <address Axxx>, <value V> represents

Core # writes value V into address Axxx.

Cx.y represents cache line y in core x.

Memory A100, 0000 → 0100 represents the value in memory location A100 is updated from 0000 to 0100.

- (1) Action: C0, R, A10C
- (2) Action: C1, W, A104, 0204
- (3) Action: C0, W, A118, 0308



MSI protocol

The initial cache and memory state for a shared memory multi-processor system are shown in the following figure. Each core processor has a **4-line direct-mapped write back** cache. Basic **write invalidation snooping protocol** is used. Please show the action results (ie. updated parts in the caches and memory) for the three action as that in the example, assuming that all the actions are applied to the initial cache and memory states in the figure. In cache state, I means Invalid, S means Shared, M means Modified.

Action specification format:

C#, R, <address Axxx> represents

Core # reads address Axxx.

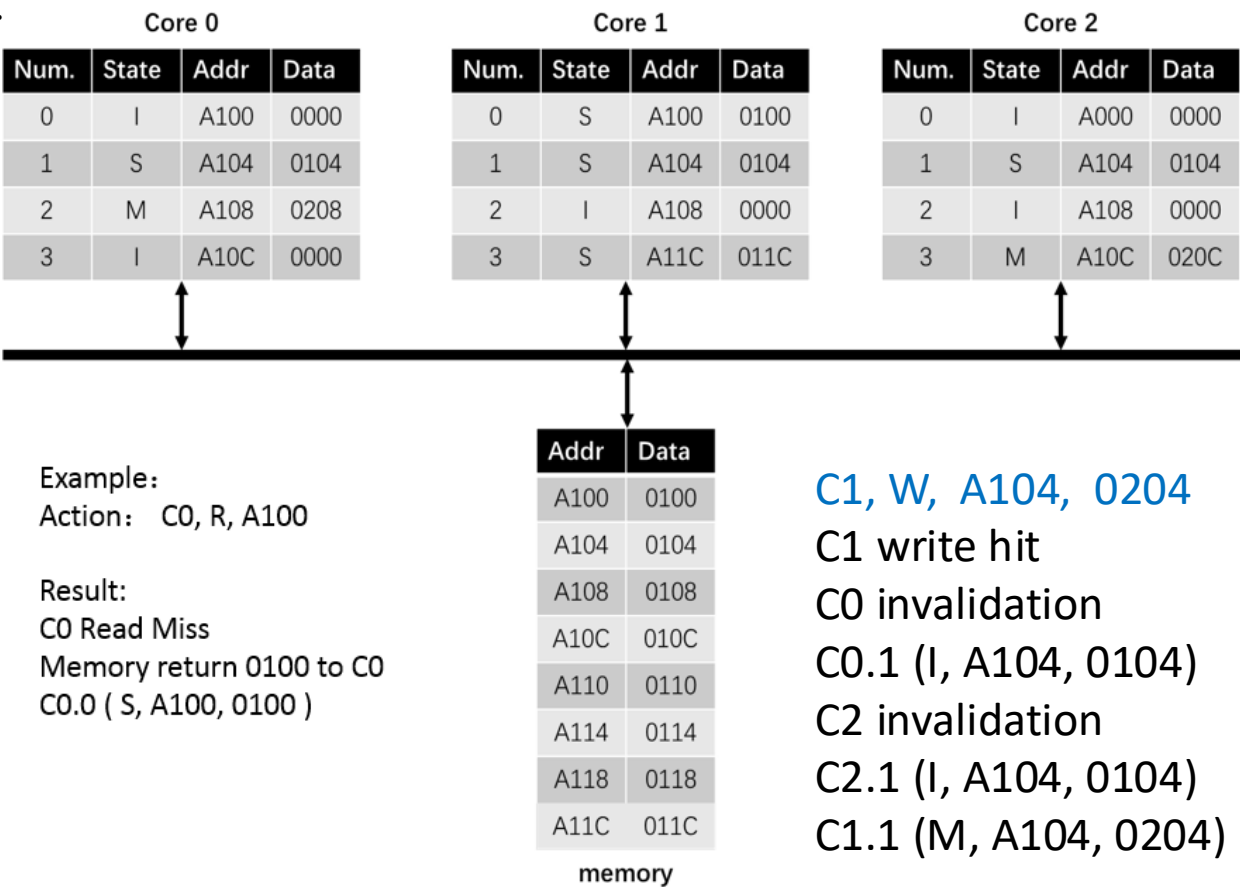
C#, W, <address Axxx>, <value V> represents

Core # writes value V into address Axxx.

Cx.y represents cache line y in core x.

Memory A100, 0000 → 0100 represents the value in memory location A100 is updated from 0000 to 0100.

- (1) Action: C0, R, A10C
- (2) Action: C1, W, A104, 0204
- (3) Action: C0, W, A118, 0308



MSI protocol

The initial cache and memory state for a shared memory multi-processor system are shown in the following figure. Each core processor has a **4-line direct-mapped write back** cache. Basic **write invalidation snooping protocol** is used. Please show the action results (ie. updated parts in the caches and memory) for the three action as that in the example, assuming that all the actions are applied to the initial cache and memory states in the figure. In cache state, I means Invalid, S means Shared, M means Modified.

Action specification format:

C#, R, <address Axxx> represents

Core # reads address Axxx.

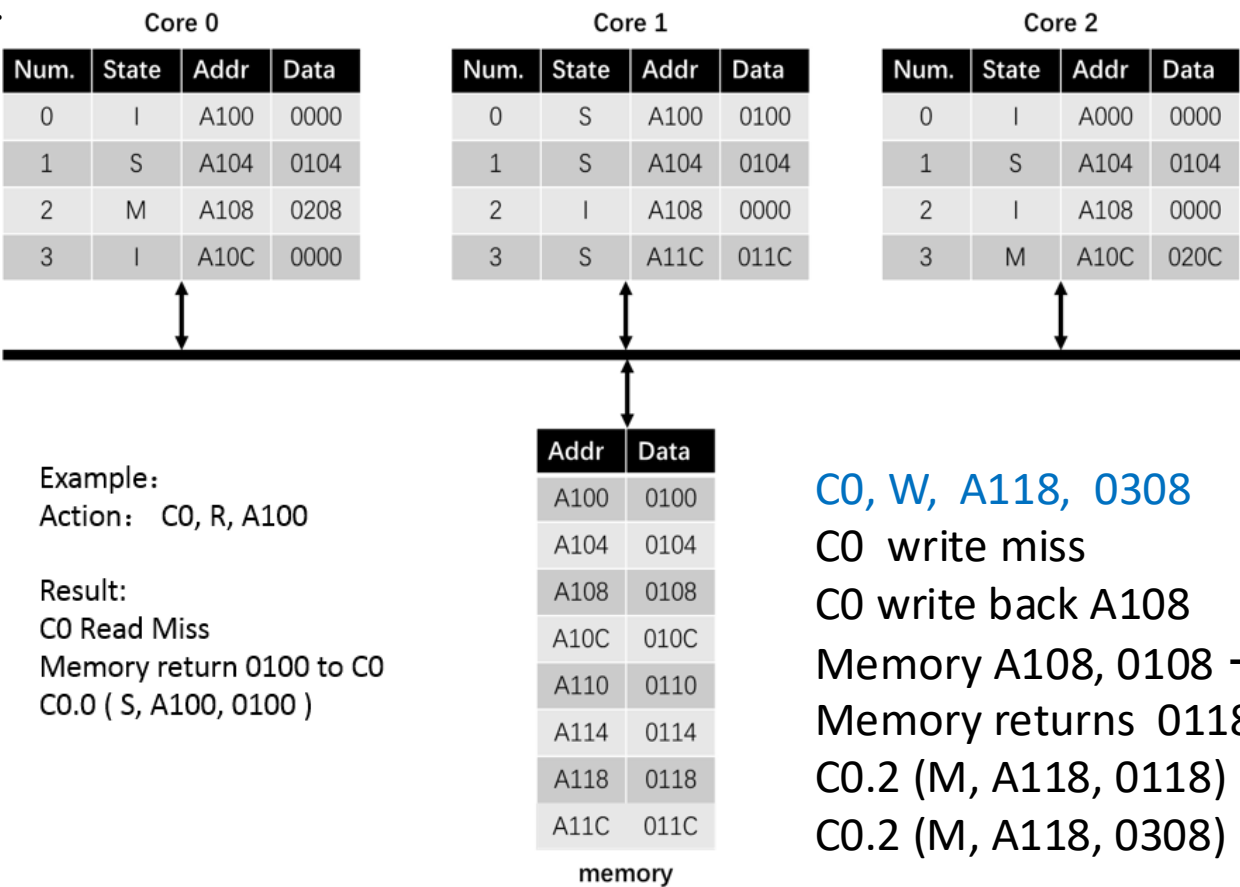
C#, W, <address Axxx>, <value V> represents

Core # writes value V into address Axxx.

Cx.y represents cache line y in core x.

Memory A100, 0000 → 0100 represents the value in memory location A100 is updated from 0000 to 0100.

- (1) Action: C0, R, A10C
- (2) Action: C1, W, A104, 0204
- (3) Action: C0, W, A118, 0308



MSI Protocol Extensions

- **MESI**

exclusive: indicates when a cache block is resident only in a single cache but is clean

exclusive->read by others->shared

exclusive->write->modified

MESI writes exclusive to modified silently, without broadcast on bus

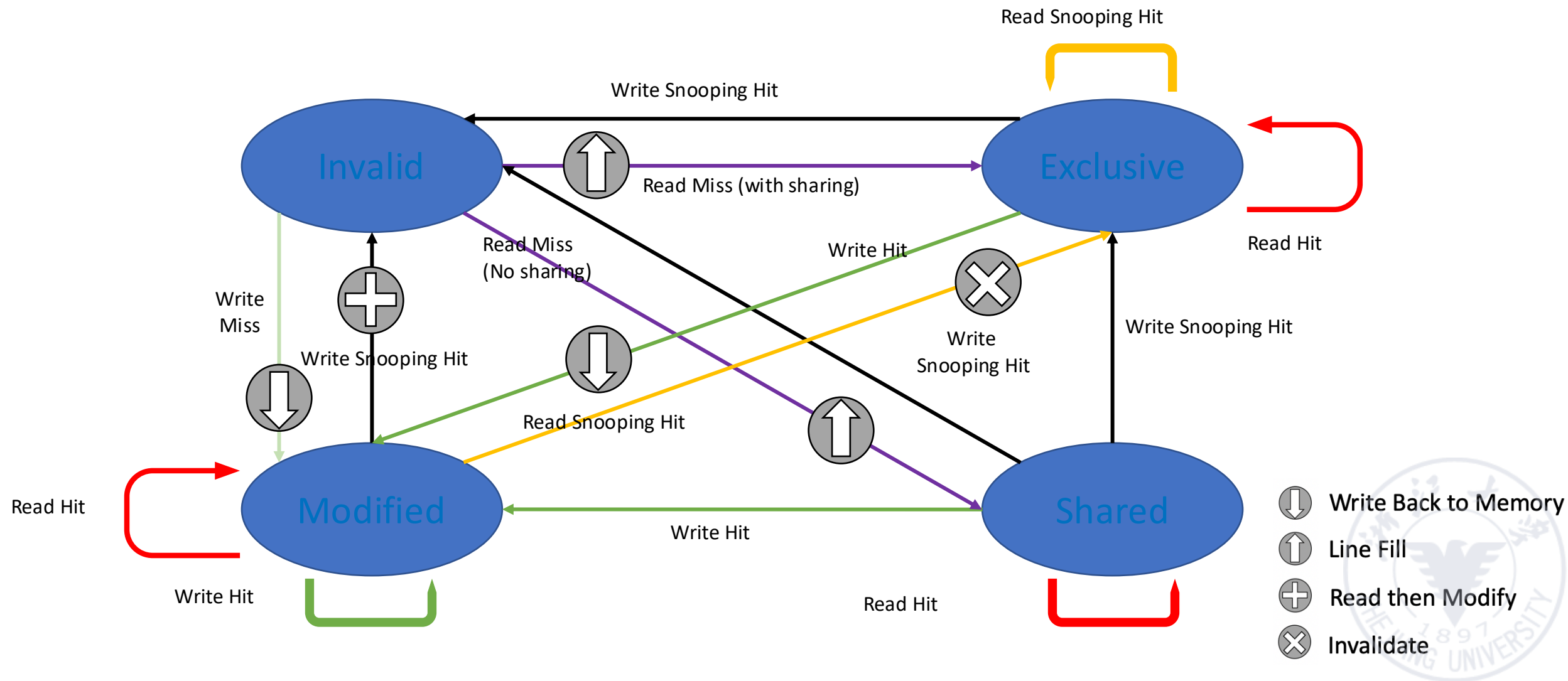


MSI Protocol Extensions (write back)

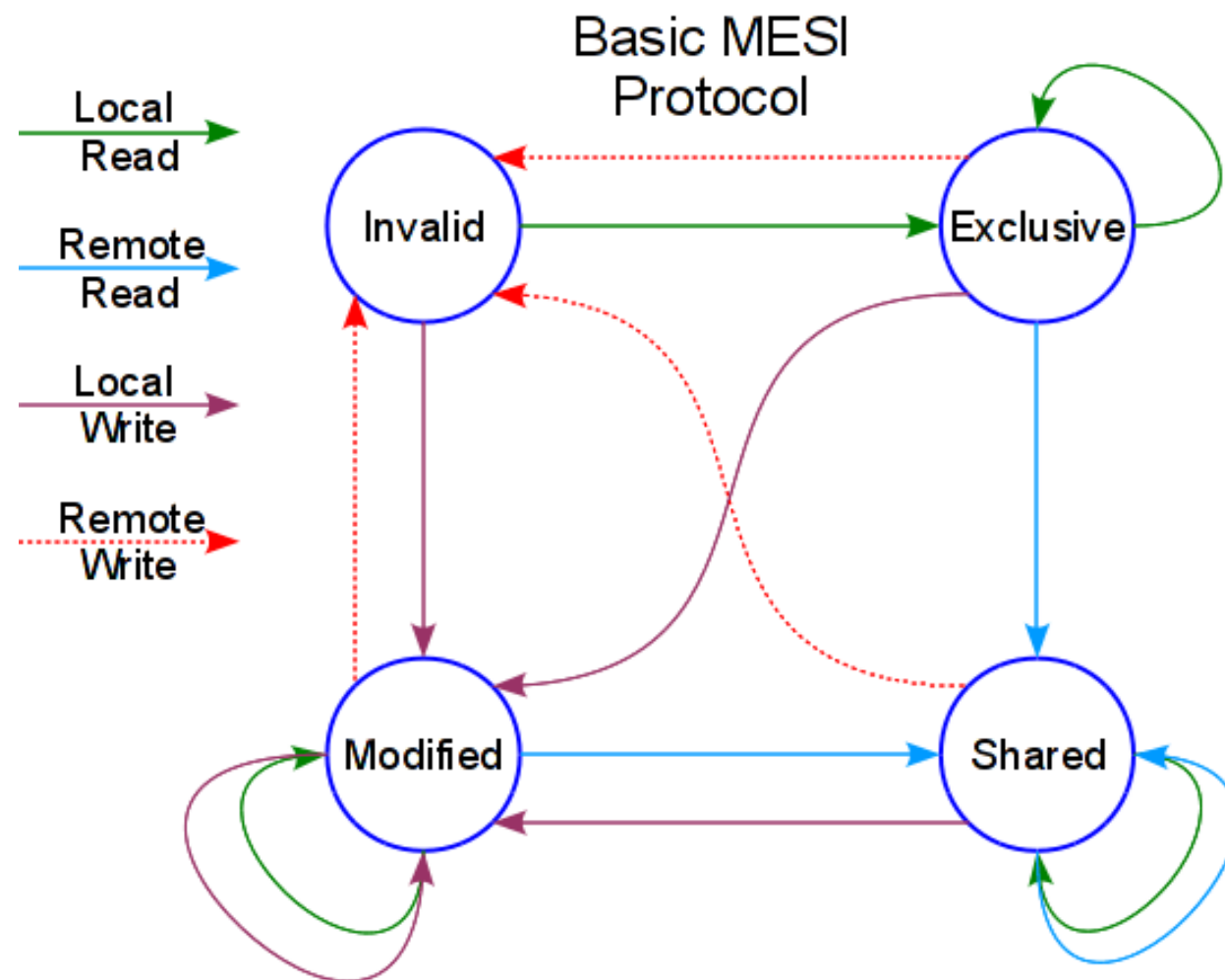
- MESI protocol: It is named after the initial letter of the four states used in the protocol. Each item in this protocol is in one of the following four states:
 - Invalid: The data contained in the cache item is invalid.
 - Shared: This row of data exists in multiple cache items, and the data in the memory is the latest.
 - Exclusive: No other cache items include this row of data, and the data in memory is the latest.
 - Modified: The data of the item is valid, but the data in the memory is invalid, and there is no copy of the data in other cache items.



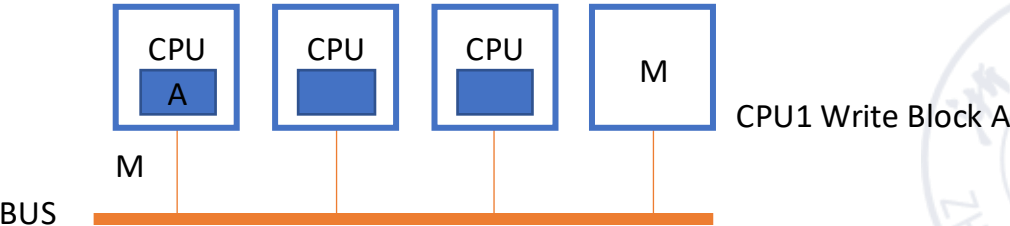
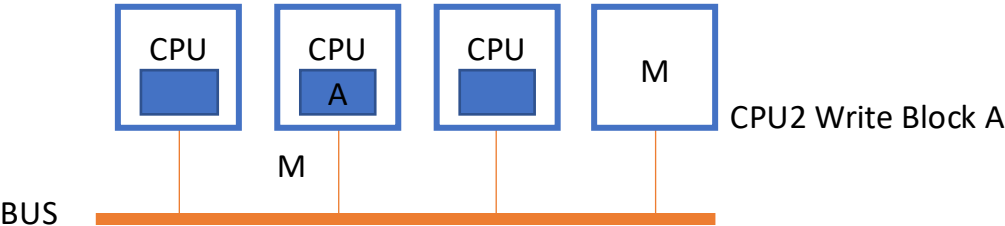
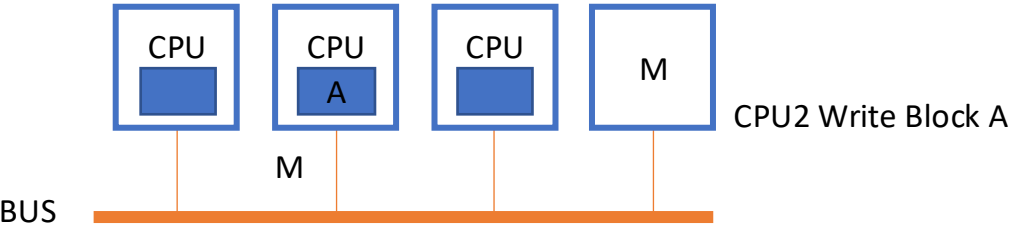
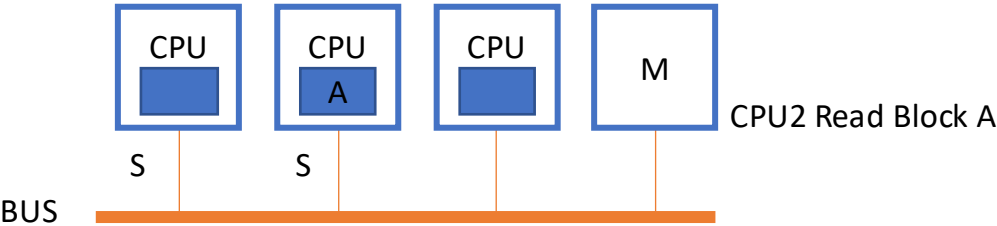
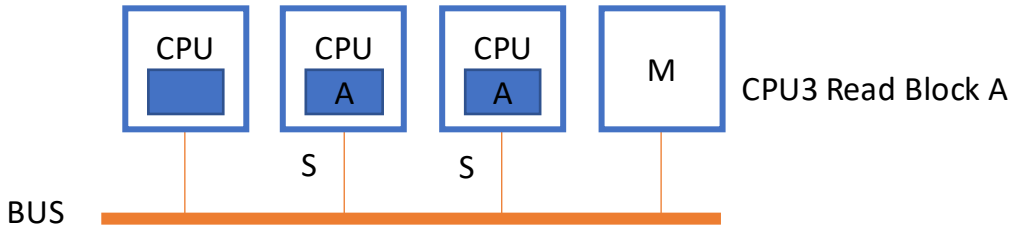
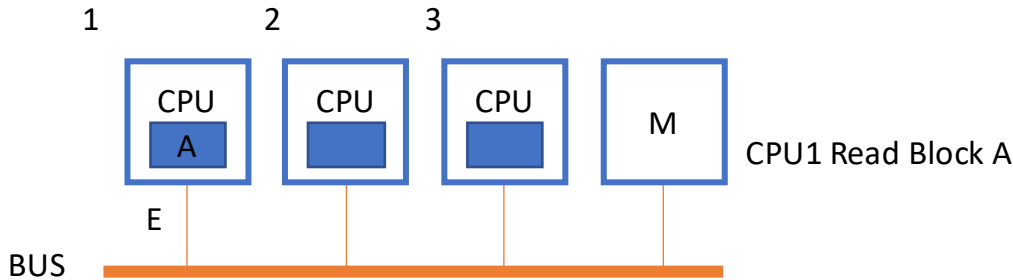
MESI protocol state transition rules



MESI protocol state transition rules

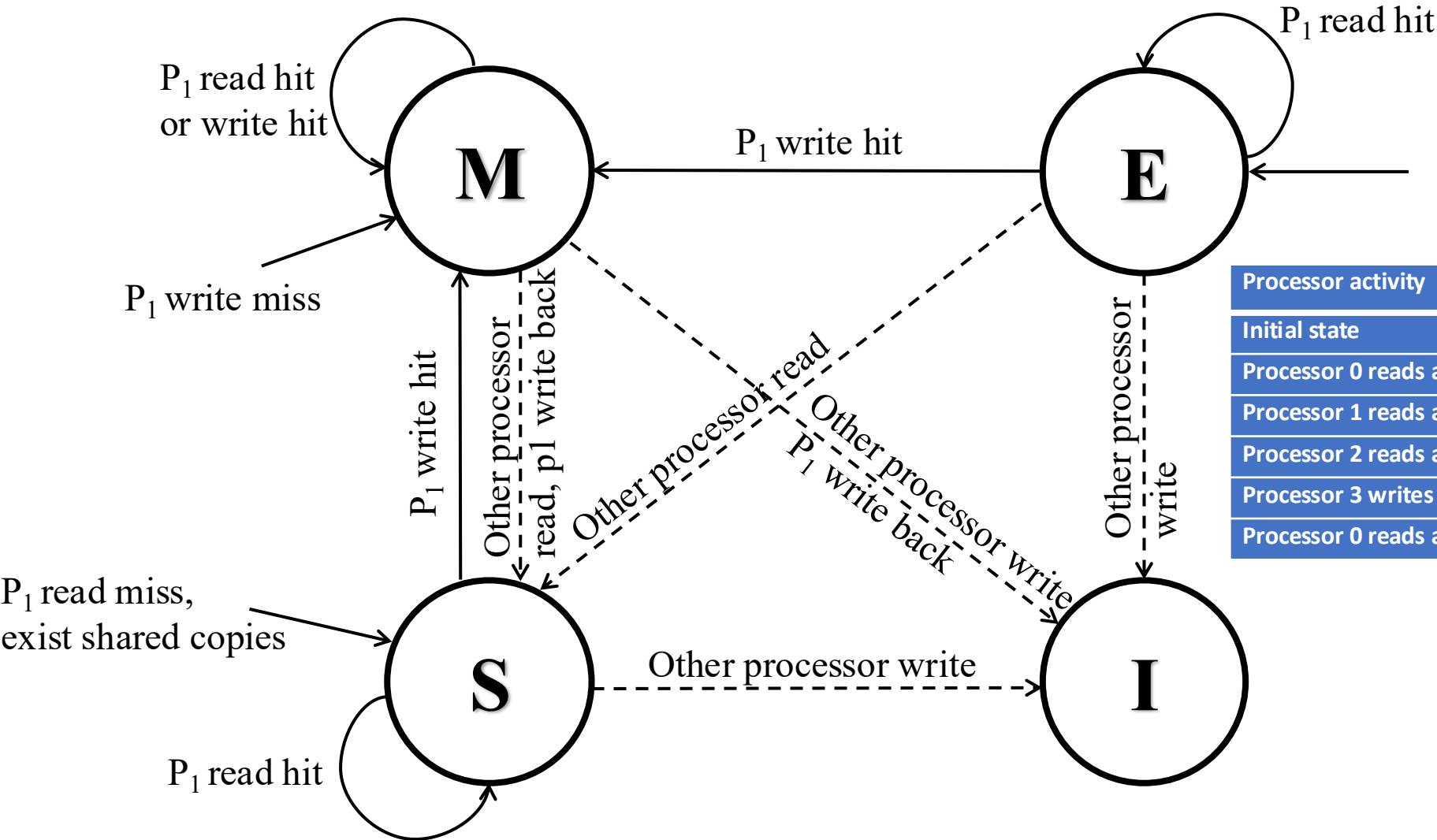


MESI protocol working process



Cache state diagram in processor P₁ with the state transitions induced by the local processor shown in solid lines and by the remote processor shown in dashed lines. Local read/write misses are merged to simplify the diagram (For local read miss, if there exist shared copies in other caches, local copy set to S or else set to E).

MESI protocol



Processor activity	0	1	2	3
Initial state	I	I	I	I
Processor 0 reads a				
Processor 1 reads a				
Processor 2 reads a				
Processor 3 writes a				
Processor 0 reads a				

MSI Extensions

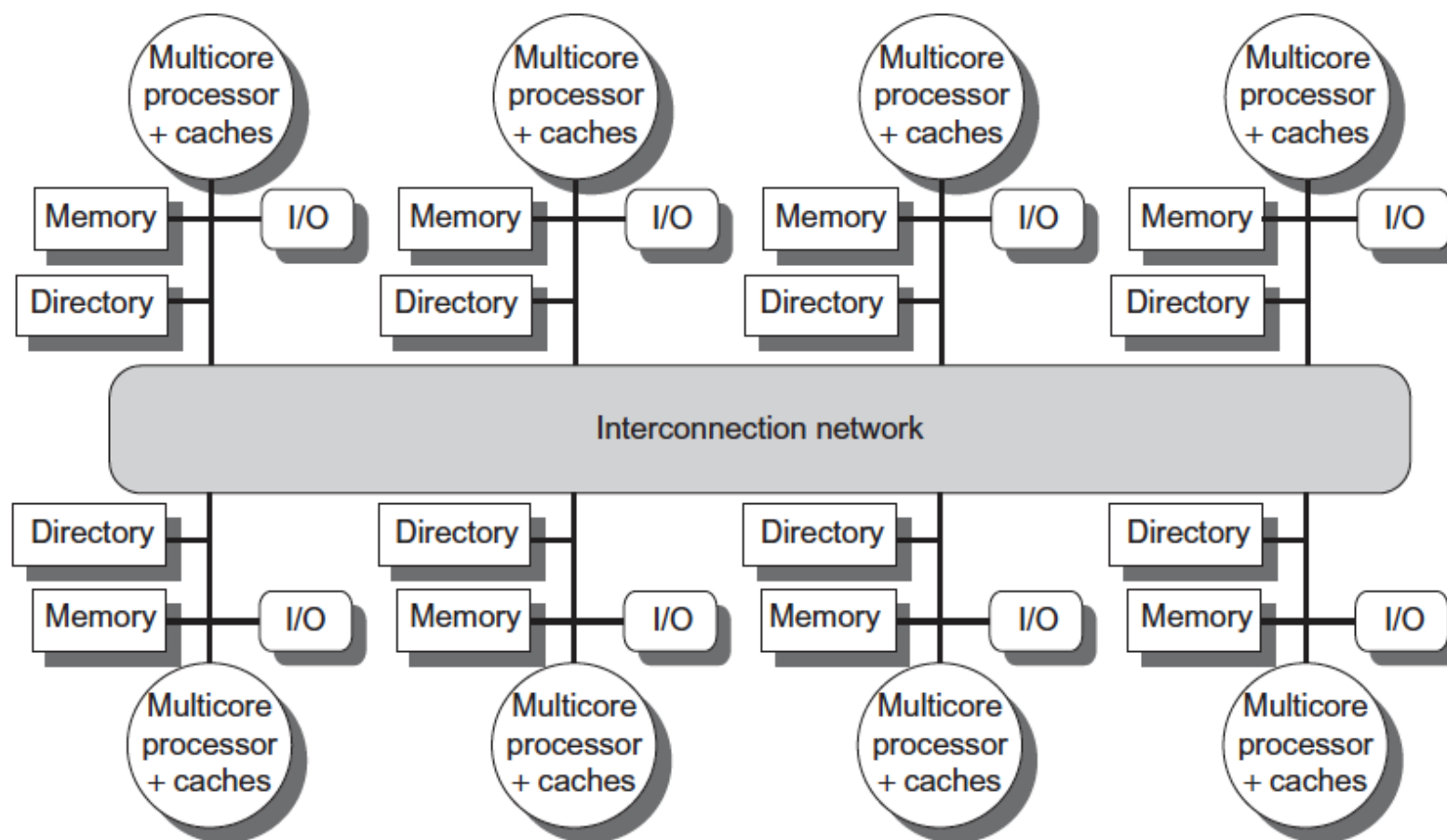
- **MOESI**

owned: indicates that the associated block is owned by that cache and out-of-date in memory

Modified -> Owned without writing the shared block to memory



Directory protocol



A directory is added to each node to implement cache coherence in a distributed-memory multiprocessor. A directory keeps the state of every block that may be cached. Information in the directory includes which caches have copies of the block, whether it is dirty, and so on.

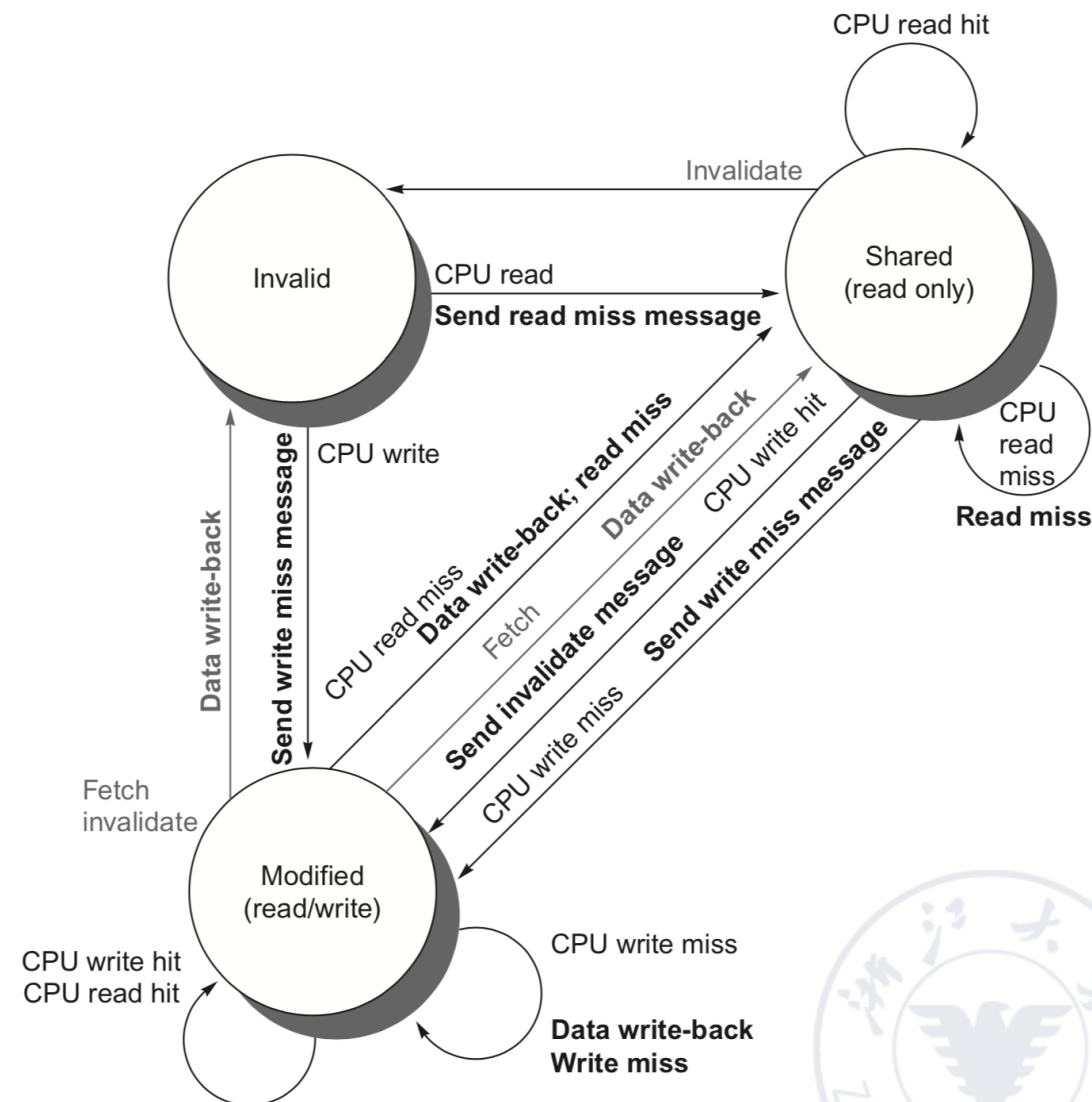
Directory protocol

- For each block, maintain state:
 - *Shared*
 - One or more nodes have the block cached, value in memory is up-to-date
 - Set of node IDs
 - *Invalid*
 - *Modified*
 - Exactly one node has a copy of the cache block, value in memory is out-of-date
 - Owner node ID
- Directory maintains block states and sends invalidation messages



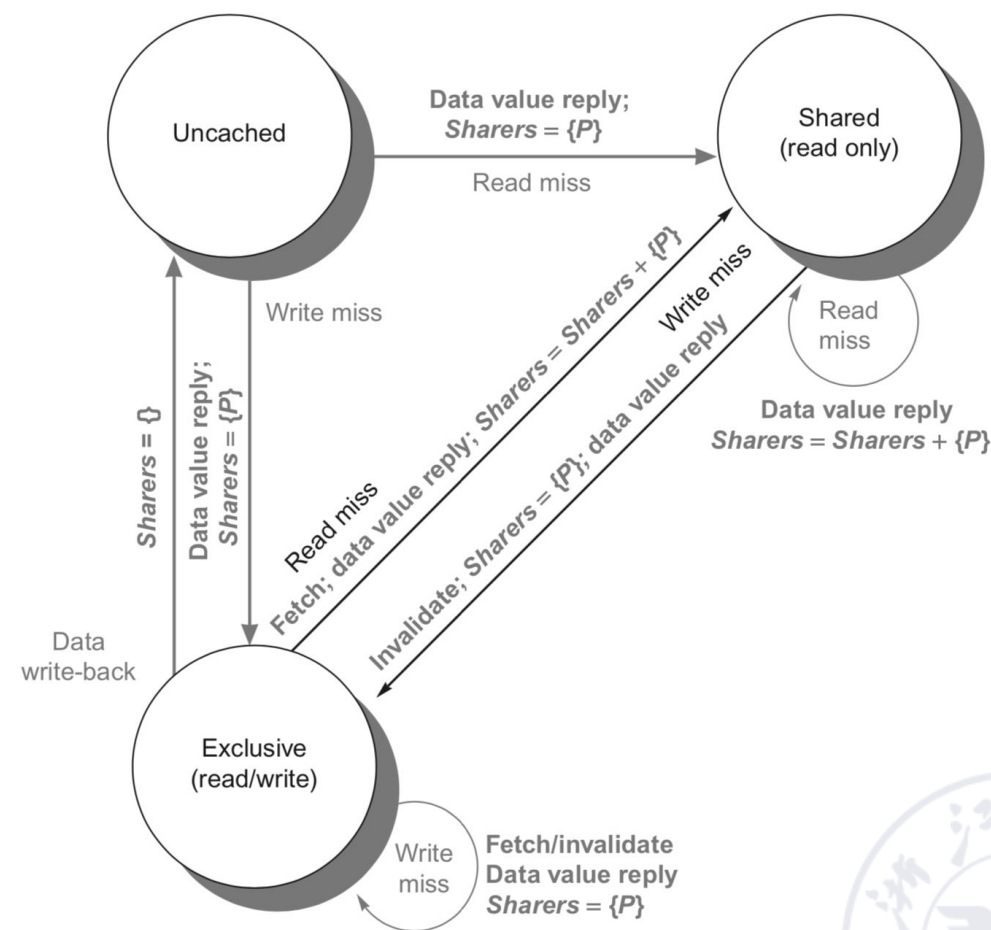
Directory protocol

- State transition diagram for *an individual cache block* in a directory-based system.
- Requests by the local processor are shown in black, and those from the home directory are shown in gray.



Directory protocol

- The state transition diagram for *the directory* has the same states and structure as the transition diagram for an individual cache.
- All actions are in gray because they are all externally caused. Bold indicates the action taken by the directory in response to the request.



Directory protocol

- For **uncached** block:
 - Read miss
 - Requesting node is sent the requested data and is made the only sharing node, block is now shared
 - Write miss
 - The requesting node is sent the requested data and becomes the sharing node, block is now exclusive
- For **shared** block:
 - Read miss
 - The requesting node is sent the requested data from memory, node is added to sharing set
 - Write miss
 - The requesting node is sent the value, all nodes in the sharing set are sent invalidate messages, sharing set only contains requesting node, block is now exclusive



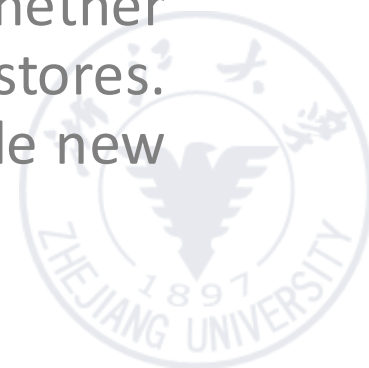
Directory protocol

- For **exclusive** block:
 - Read miss
 - The owner is sent a data fetch message, block becomes shared, owner sends data to the directory, data written back to memory, sharers set contains old owner and requestor
 - Data write back
 - Block becomes uncached, sharer set is empty
 - Write miss
 - Message is sent to old owner to invalidate and send the value to the directory, requestor becomes new owner, block remains exclusive



Memory Consistency and Cache Coherence

- Memory Consistency **Need Memory Consistency Model**
 - When a written value will be returned by a read
 - If a processor writes location A followed by location B, any processor that sees the new value of B must also see the new value of A
- Cache Coherence Need Cache Coherence Protocol
 - All reads by any processor must return the most recently written value
 - Writes to the same location by any two processors are seen in the same order by all processors
 - Correct coherence ensures that a programmer cannot determine whether and where a system has caches by analyzing the results of loads and stores. This is because correct coherence ensures that the caches never enable new or different *functional* behavior.



Memory Consistency

- Example:

Processor 1:

A=0

...

A=1

if (B==0) ...

Processor 2:

B=0

...

B=1

if (A==0) ...

- Sequential consistency (reduces potential performance):
 - Result of execution should be the same as long as:
 - Accesses on each processor were kept in order
 - Accesses on different processors were arbitrarily interleaved



Relaxed Consistency Models

- Key idea: allow reads and writes to complete out of order, but to use synchronization operations to enforce ordering



Relaxed Consistency Models

- Rules:
 - $X \rightarrow Y$
 - Operation X must complete before operation Y is done
 - Sequential consistency requires:
 - $R \rightarrow W, R \rightarrow R, W \rightarrow R, W \rightarrow W$
 - Relax $W \rightarrow R$
 - “Total store ordering”
 - Relax $W \rightarrow W$
 - “Partial store order”
 - Relax $R \rightarrow W$ and $R \rightarrow R$
 - “Weak ordering” and “release consistency”



Domain-Specific Architectures

- Moore's Law enabled:
 - Deep memory hierarchy
 - Wide SIMD units
 - Deep pipelines
 - Branch prediction
 - Out-of-order execution
 - Speculative prefetching
 - Multithreading
 - Multiprocessing
- Objective:
 - Extract performance from software that is oblivious to architecture



Guidelines for DSAs

- Use dedicated memories to minimize data movement
- Invest resources into more arithmetic units or bigger memories
- Use the easiest form of parallelism that matches the domain
- Reduce data size and type to the simplest needed for the domain
- Use a domain-specific programming language



Example: Convolutional Neural Network

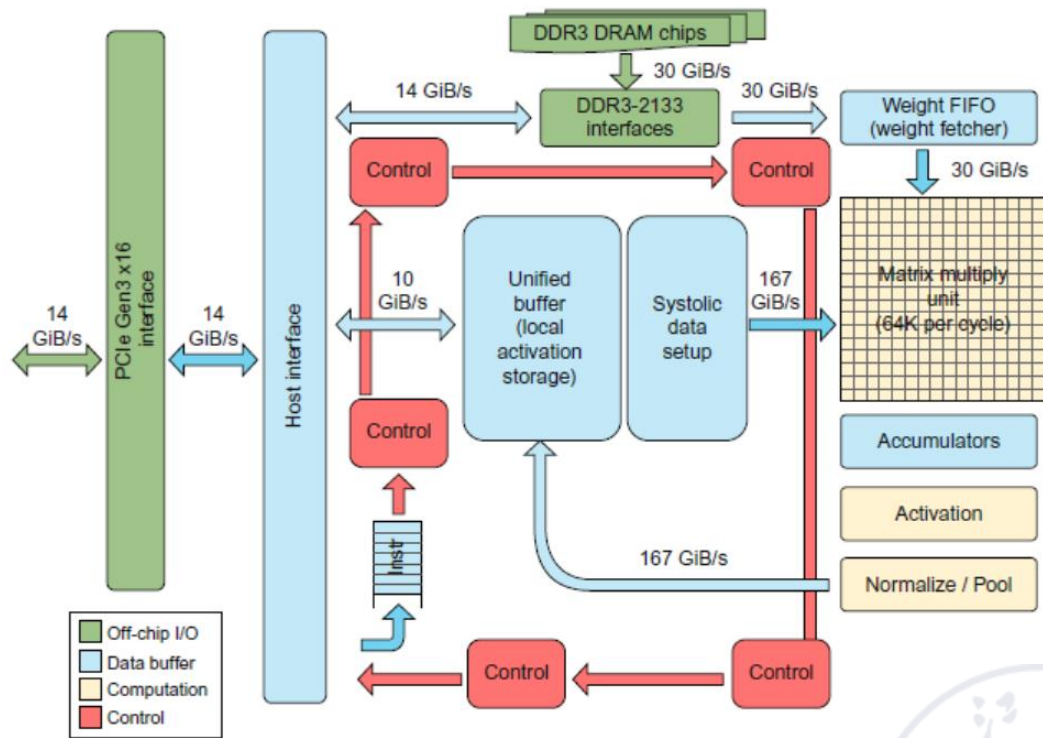
Abstraction in Computer Architecture View

- Batches:
 - Reuse weights once fetched from memory across multiple inputs
 - Increases operational intensity
- Quantization
 - Use 8- or 16-bit fixed point
- Summary:
 - Need the following kernels:
 - *Matrix-vector multiply*
 - *Matrix-matrix multiply*
 - *ReLU*
 - *Sigmoid*
 - ...



Tensor Processing Unit

- Google's DNN ASIC
- 256 x 256 8-bit matrix multiply unit
- Large software-managed scratchpad
- Coprocessor on the PCIe bus

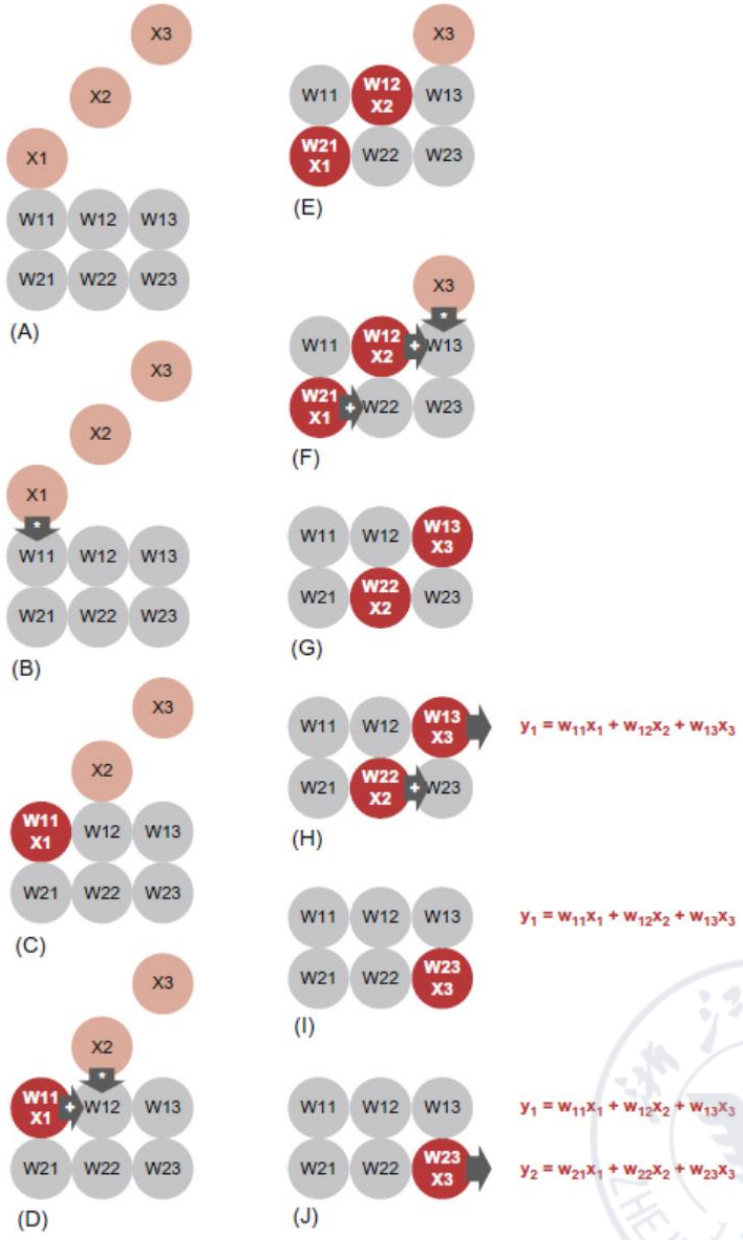
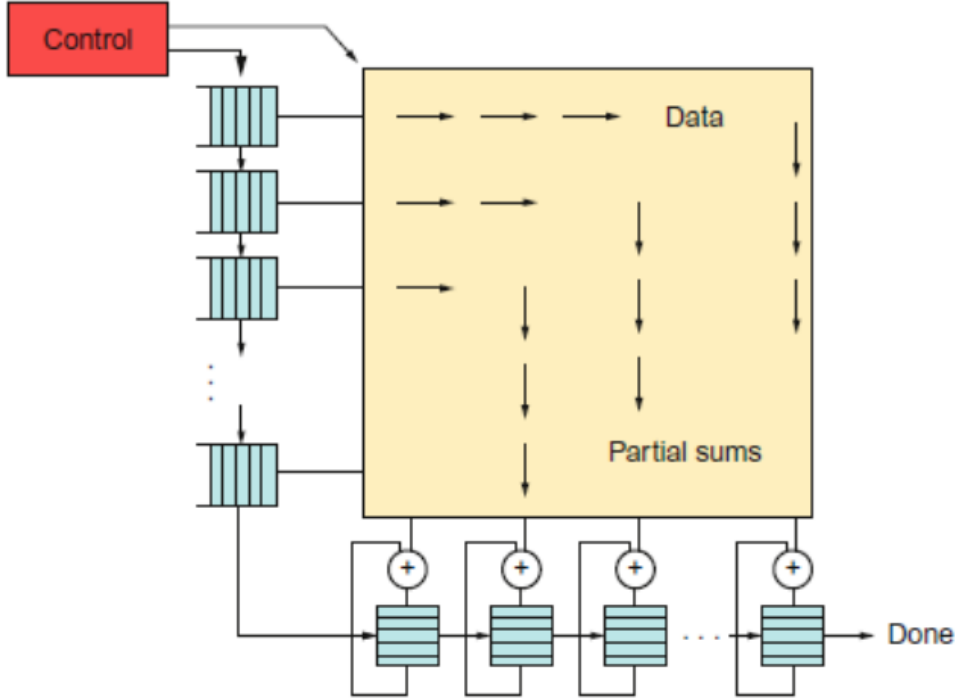


TPU ISA

- **Read_Host_Memory**
 - Reads memory from the CPU memory into the unified buffer
- **Read_Weights**
 - Reads weights from the Weight Memory into the Weight FIFO as input to the Matrix Unit
- **MatrixMatrixMultiply/Convolve**
 - Perform a matrix-matrix multiply, a vector-matrix multiply, an element-wise matrix multiply, an element-wise vector multiply, or a convolution from the Unified Buffer into the accumulators
 - takes a variable-sized $B \times 256$ input, multiplies it by a 256×256 constant input, and produces a $B \times 256$ output, taking B pipelined cycles to complete
- **Activate**
 - Computes activation function
- **Write_Host_Memory**
 - Writes data from unified buffer into host memory



Matrix Calculations in TPU



The TPU and the Guidelines

- Use dedicated memories
 - 24 MiB dedicated buffer, 4 MiB accumulator buffers
- Invest resources in arithmetic units and dedicated memories
 - 60% of the memory and 250X the arithmetic units of a server-class CPU
- Use the easiest form of parallelism that matches the domain
 - Exploits 2D SIMD parallelism
- Reduce the data size and type needed for the domain
 - Primarily uses 8-bit integers
- Use a domain-specific programming language
 - Uses TensorFlow



Summary

